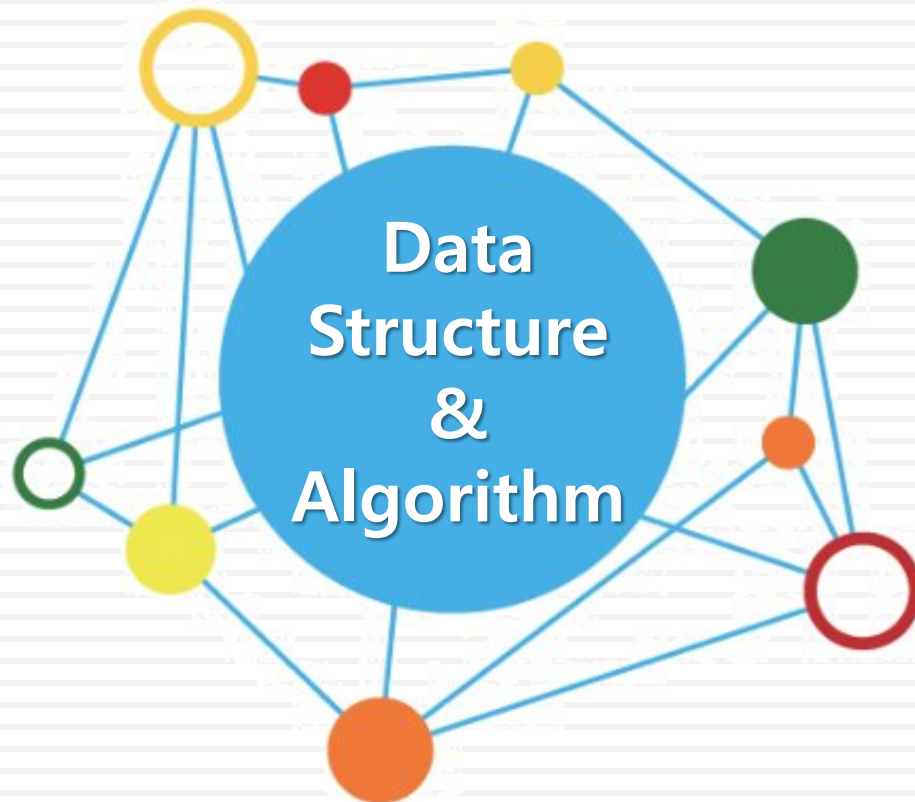


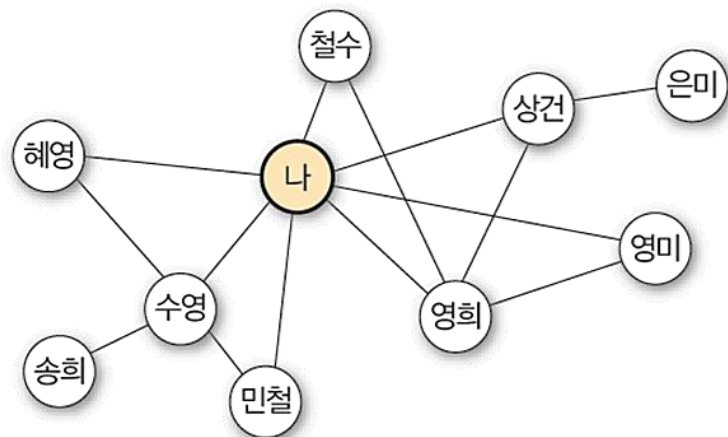
# 데이터 구조론



유길현  
그래프(Graph)

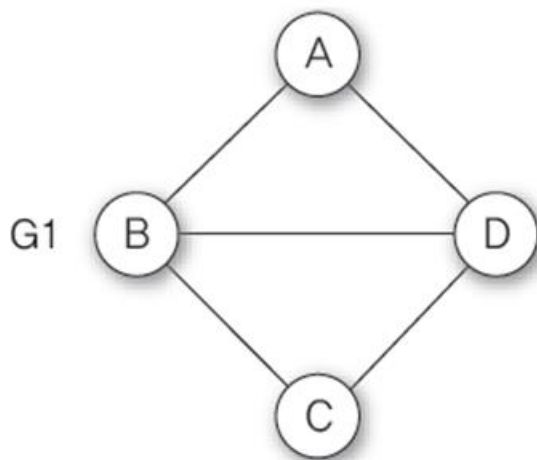
## ❖ 그래프(Graph)란

- 연결되어 있는 원소 사이의 n:n 관계를 표현하는 자료구조
- 그래프 G
  - 객체를 나타내는 **정점(vertex)**과 객체를 연결하는 **간선(edge)**의 집합
  - $G = (V, E)$ 
    - V는 그래프에 있는 정점들의 집합
    - E는 정점을 연결하는 간선들의 집합



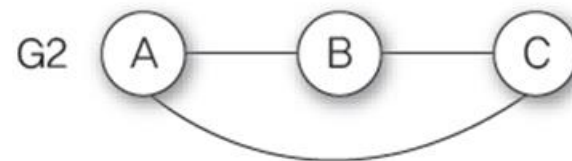
## ❖ 그래프의 종류

- 무방향 그래프(undirected graph)
  - 두 정점을 연결하는 간선에 방향이 없는 그래프
  - 정점  $V_i$ 와 정점  $V_j$ 를 연결하는 간선을  $(V_i, V_j)$ 로 표현
    - $(V_i, V_j)$ 와  $(V_j, V_i)$ 는 같은 간선을 의미



$$V(G1) = \{A, B, C, D\}$$

$$E(G1) = \{(A,B), (A, D), (B, C), (B, D), (C, D)\}$$

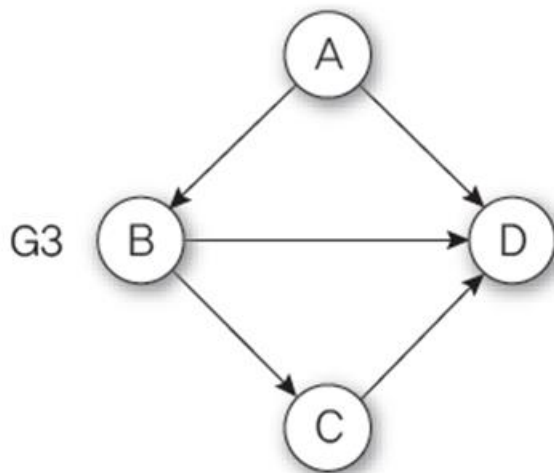


$$V(G2) = \{A, B, C\}$$

$$E(G2) = \{(A,B), (A, C), (B, C)\}$$

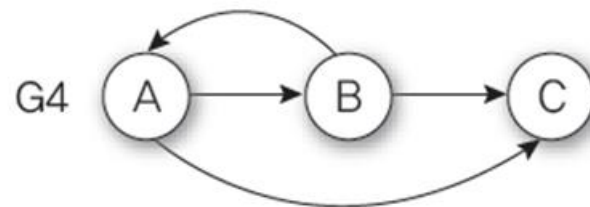
## ❖ 그래프의 종류

- 방향 그래프(directed graph), 다이그래프(digraph)
  - 간선에 방향이 있는 그래프
  - 정점  $V_i$ 와 정점  $V_j$ 를 연결하는 간선 즉,  $V_i \rightarrow V_j$ 를  $\langle V_i, V_j \rangle$ 로 표현
    - $V_i$ 를 꼬리(tail),  $V_j$ 를 머리(head)라고 한다.
    - $\langle V_i, V_j \rangle$ 와  $\langle V_j, V_i \rangle$ 는 서로 다른 간선을 의미



$V(G1) = \{A, B, C, D\}$

$E(G1) = \{\langle A, B \rangle, \langle A, D \rangle, \langle B, C \rangle, \langle B, D \rangle, \langle C, D \rangle\}$

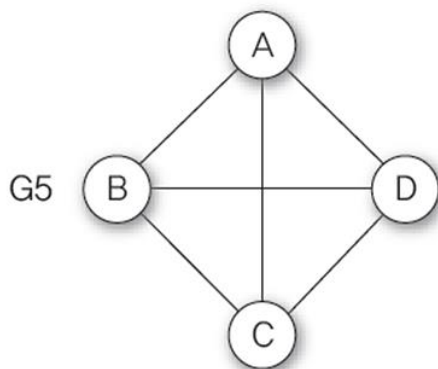


$V(G2) = \{A, B, C\}$

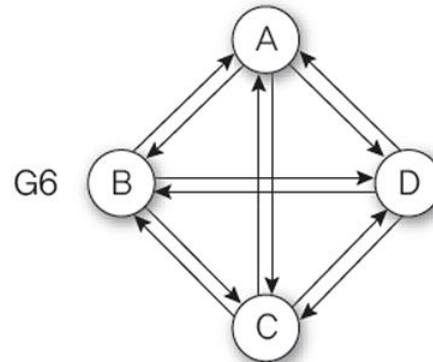
$E(G1) = \{\langle A, B \rangle, \langle A, C \rangle, \langle B, A \rangle, \langle B, C \rangle\}$

## ❖ 그래프의 종류

- 완전 그래프(complete graph)
  - 각 정점에서 다른 모든 정점을 연결하여 최대한 많은 간선 수를 가진 그래프
  - 정점이  $n$ 개인 무방향 그래프에서 최대의 간선 수 :  $n(n-1)/2$ 개
  - 정점이  $n$ 개인 방향 그래프의 최대 간선 수 :  $n(n-1)$ 개
  - 완전 그래프의 예



G5는 정점의 개수가 4개인 무 방향 그래프이므로 완전 그래프가 되려면  $4(4-1)/2=6$ 개의 간선 연결



G6은 정점의 개수가 4개인 방향 그래프이므로 완전 그래프가 되려면  $4(4-1)=12$ 개의 간선 연결

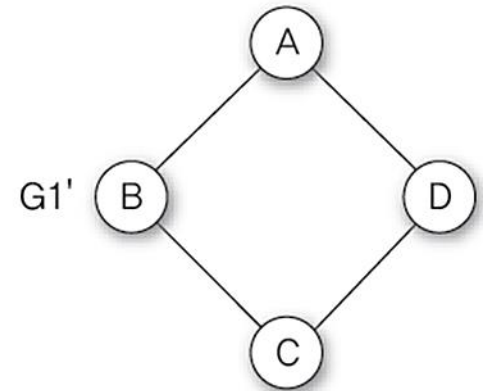
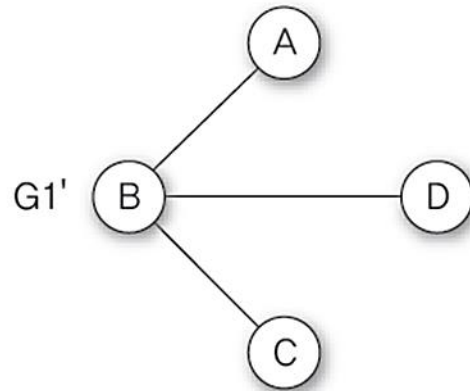
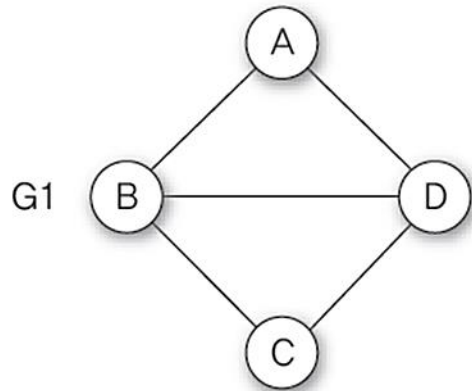
## ❖ 그래프의 종류

### ■ 부분 그래프(subgraph)

- 원래의 그래프에서 정점이나 간선을 일부만 제외하여 만든 그래프
- 그래프  $G$ 와 부분 그래프  $G'$ 의 관계

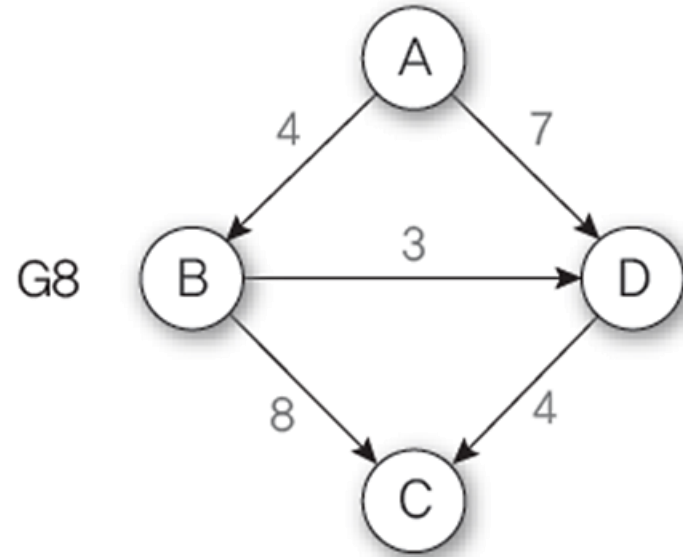
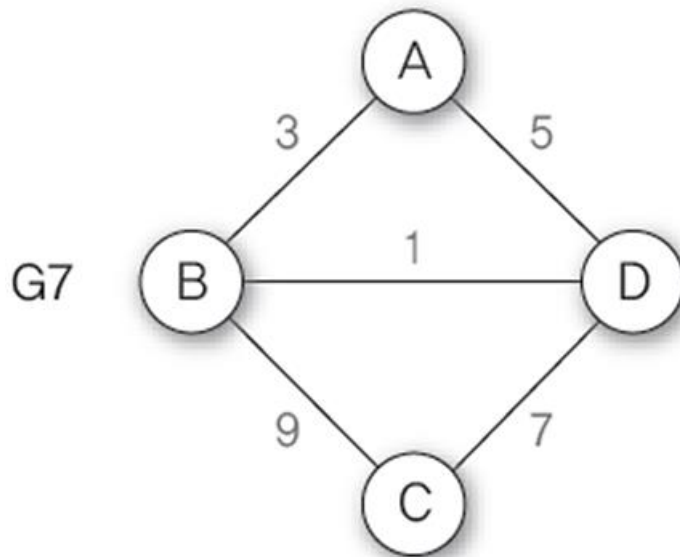
$$V(G') \subseteq V(G), E(G') \subseteq E(G)$$

- 그래프  $G_1$ 에 대한 부분 그래프의 예



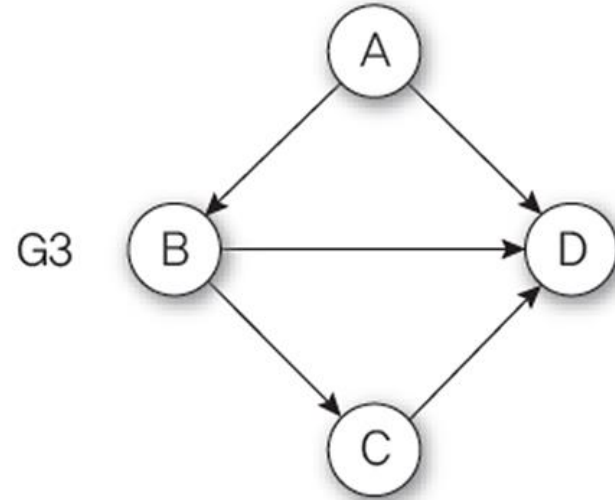
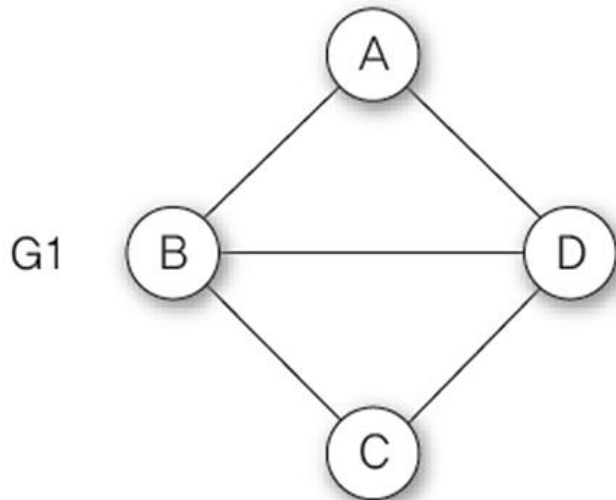
## ❖ 그래프의 종류

- 가중 그래프(weight), 네트워크(network)
  - 정점을 연결하는 간선에 가중치(weight)를 할당한 그래프



## ❖ 그래프 관련 용어

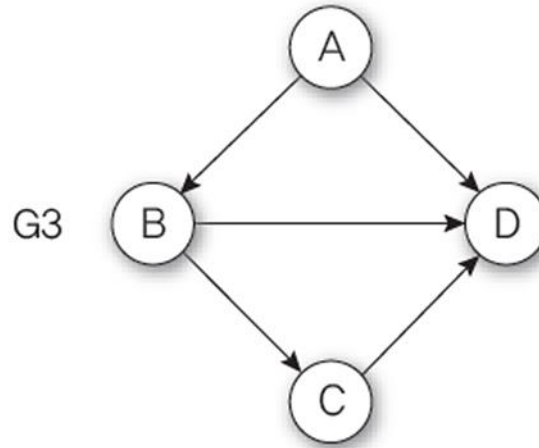
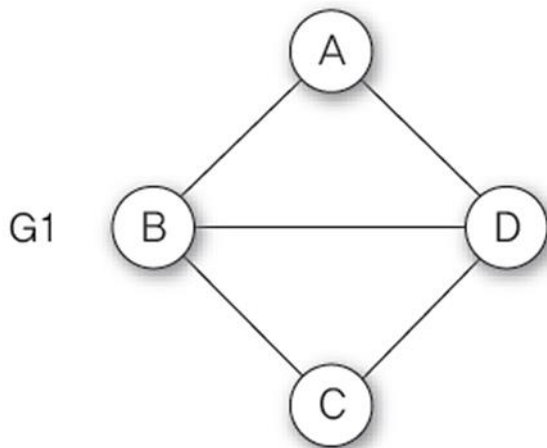
- 그래프에서 두 정점  $V_i$ 와  $V_j$ 를 연결하는 간선  $(V_i, V_j)$ 가 있을 때, 두 정점  $V_i$ 와  $V_j$ 를 인접(adjacent)되어 있다고 하고, 간선  $(V_i, V_j)$ 는 정점  $V_i$ 와  $V_j$ 에 부속(incident)되어있다고 함
- 그래프  $G_1$ 에서 정점 A와 인접한 정점은 B와 D이고, 정점 A에 부속되어 있는 간선은  $(A,B)$ 와  $(A,D)$





## ❖ 그래프 용어

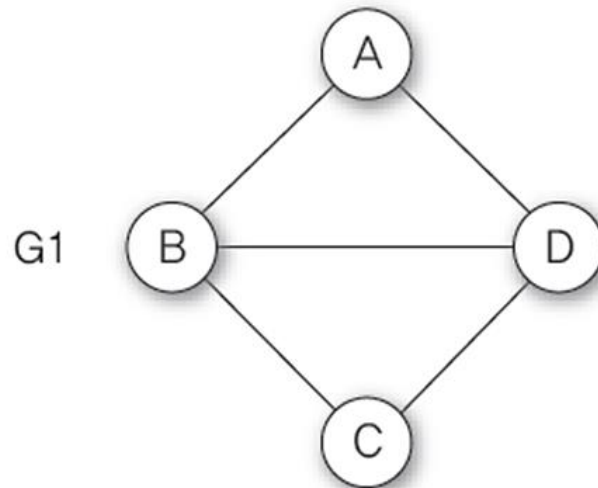
- 차수(degree) – 정점에 부착되어있는 간선의 수
  - 무방향 그래프 G1에서 정점 A의 차수는 2, 정점 B의 차수는 3
  - 방향 그래프의 정점의 차수 = 진입차수 + 진출차수
    - 진입차수(in-degree) : 정점을 머리로 하는 간선의 수
    - 진출차수(out-degree) : 정점을 꼬리로 하는 간선의 수
    - 방향 그래프 G3에서 정점 B의 진입차수는 1, 진출차수는 2  
정점 B의 전체 차수는 (진입차수 + 진출차수) 이므로 3임



## ❖ 그래프 용어

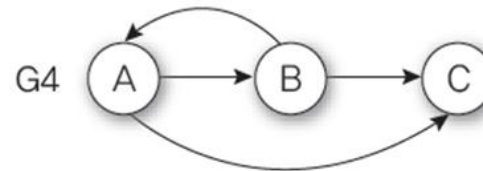
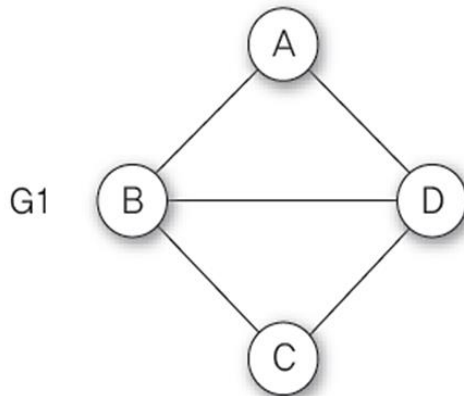
### ■ 경로(path)

- 그래프에서 간선을 따라 갈 수 있는 길을 순서대로 나열한 것 즉, 정점  $V_i$ 에서  $V_j$ 까지 간선으로 연결된 정점을 순서대로 나열한 리스트
  - 무방향 그래프  $G_1$ 에서 정점 A에서 정점 C까지는 A-B-C 경로와 A-B-D-C 경로, A-D-C 경로, 그리고 A-D-B-C 경로가 있음
- 경로길이(path length) - 경로를 구성하는 간선의 수



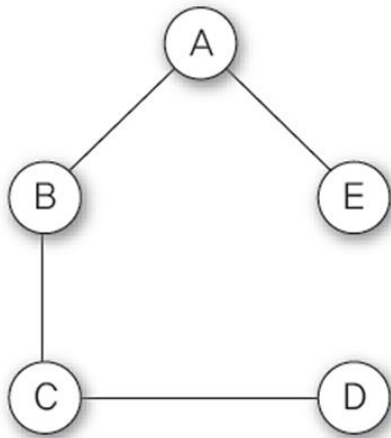
## ❖ 그래프 용어

- 단순경로(simple path) : 모두 다른 정점으로 구성된 경로
  - 그래프 G1에서 정점 A에서 정점 C까지의 경로 A-B-C는 단순 경로이고, 경로 A-B-D-A-B-C는 단순경로가 아님
- 사이클(cycle) : 단순경로 중에서 경로의 시작 정점과 마지막 정점이 같은 경로
  - 그래프 G1에서 단순경로 A-B-C-D-A와 그래프 G4에서 단순경로 A-B-A는 사이클이 됨
- DAG(directed acyclic graph) : 방향 그래프이면서 사이클이 없는 그래프

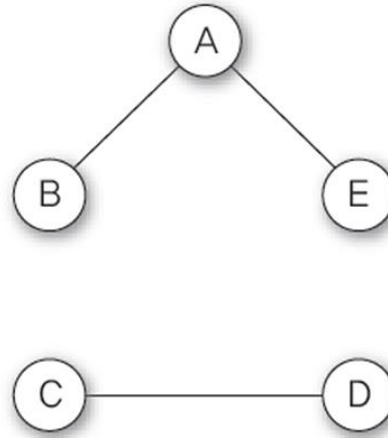


## ❖ 그래프 용어

- 연결 그래프(connected graph)
  - 서로 다른 모든 쌍의 정점들 사이에 경로가 있는 그래프 즉, 떨어져있는 정점이 없는 그래프
  - 그래프에서 두 정점  $V_i$ 에서  $V_j$ 까지의 경로가 있으면 정점  $V_i$ 와  $V_j$ 가 연결(connected)되었다고 함. 트리는 사이클이 없는 연결 그래프
- 단절 그래프(Disconnected Graph) – 연결되지 않은 정점이 있는 그래프



연결 그래프



단절 그래프

## ❖ 그래프의 추상 자료형

| ADT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 그래프의 추상 자료형 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>ADT Graph</b><br>데이터 : 공백이 아닌 정점의 집합과 간선의 집합<br>연산 :<br>$g \in \text{Graph}; u, v \in V;$<br><br>// 공백 그래프의 생성 연산<br>createGraph() ::= create an empty Graph;<br><br>// 그래프 g가 정점이 없는 공백 그래프인지를 검사하는 연산<br>isEmpty(g) ::= <b>if</b> (g have no vertex) <b>then return</b> true;<br><b>else return</b> false;<br><br>// 그래프 g에 정점 v를 삽입하는 연산<br>insertVertex(g, v) ::= insert vertex v into g;<br><br>// 그래프 g에 간선 (u,v)를 삽입하는 연산<br>insertEdge(g, u, v) ::= insert edge (u,v) into g; |             |

## ❖ 그래프의 추상 자료형

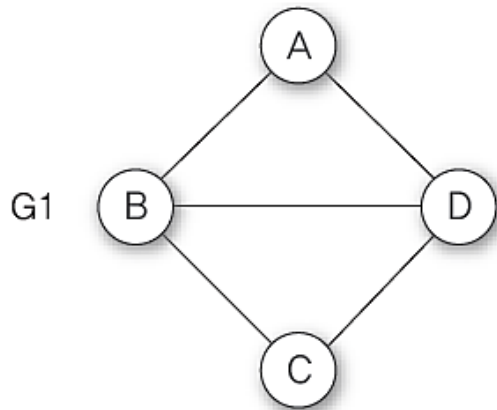
| ADT | 그래프의 추상 자료형                                                                                                                                                                                                                                                                                                                          |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <pre>// 그래프 g에서 정점 v를 삭제하고 그에 부속된 모든 간선을 삭제하는 연산 deleteVertex(g, v) ::= delete vertex v and all edges incident on v from g;  // 그래프 g에서 간선 (u,v)를 삭제하는 연산 deleteEdge(g, u, v) ::= delete edges (u,v) from g;  // 정점 v에 인접한 모든 정점을 반환하는 연산 adjacent(g, v) ::= <b>return</b> set of all vertices adjacent to v; <b>End Graph</b></pre> |

## ❖ 순차 자료 구조를 이용한 그래프의 구현 : 인접 행렬

- 행렬에 대한 2차원 배열을 사용하는 순차 자료구조 방법
- 그래프의 두 정점을 연결한 간선의 유무를 행렬로 저장
  - $n$ 개의 정점을 가진 그래프 :  $n \times n$  정방행렬
  - 행렬의 행 번호와 열 번호 : 그래프의 정점
  - 행렬 값 : 두 정점이 인접되어 있으면 1, 인접되지 않으면 0
- 무방향 그래프의 인접 행렬
  - 행  $i$ 의 합 = 열  $i$ 의 합 = 정점  $i$ 의 차수
- 방향 그래프의 인접 행렬
  - 행  $i$ 의 합 = 정점  $i$ 의 진출 차수
  - 열  $i$ 의 합 = 정점  $i$ 의 진입 차수

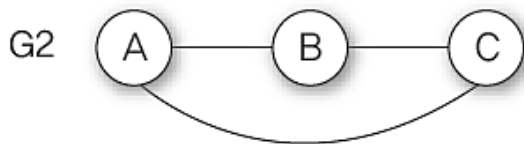
## ❖ 그래프의 인접행렬 표현

- 무방향 그래프의 인접 행렬
  - 행  $i$ 의 합 = 열  $i$ 의 합 = 정점  $i$ 의 차수



|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 |
| B | 1 | 0 | 1 | 1 |
| C | 0 | 1 | 0 | 1 |
| D | 1 | 1 | 1 | 0 |

1+0+1+1=3 정점 B의 차수



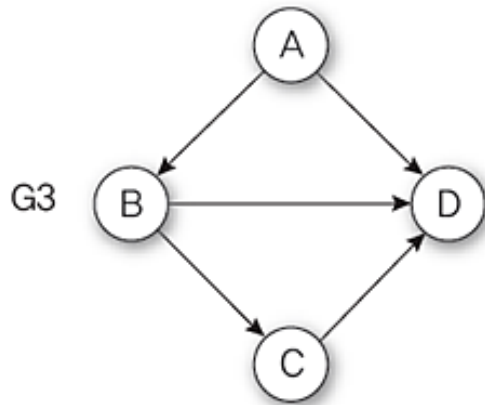
|   | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 1 | 0 | 1 |
| C | 1 | 1 | 0 |



## ❖ 그래프의 인접행렬 표현

### ■ 방향 그래프의 인접 행렬

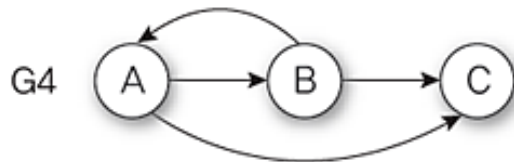
- 행  $i$ 의 합 = 정점  $i$ 의 진출 차수
- 열  $j$ 의 합 = 정점  $j$ 의 진입 차수



|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 |
| B | 0 | 0 | 1 | 1 |
| C | 0 | 0 | 0 | 1 |
| D | 0 | 0 | 0 | 0 |

$0+0+1+1=2$  정점 B의 진출 차수

$1+0+0+0=1$  정점 B의 진입 차수



|   | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 1 | 0 | 1 |
| C | 0 | 0 | 0 |

## ❖ 그래프의 인접행렬 표현하기 프로그램

- 그래프를 인접행렬로 표현하기 위한 구조체 정의

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTEX 30

// 그래프를 인접 행렬로 표현하기 위한 구조체 정의
typedef struct graphType {
    int n;          // 그래프의 정점 개수
    // 그래프에 대한 30x30의 2차원 배열
    int adjMatrix[MAX_VERTEX][MAX_VERTEX];
} graphType;
```

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 공백 그래프의 생성

```
// 공백 그래프를 생성하는 연산
void createGraph(graphType* g)
{
    int i, j;
    g->n = 0;      // 정점 개수를 0으로 초기화
    for (i = 0; i < MAX_VERTEX; i++) {
        for (j = 0; j < MAX_VERTEX; j++)
            // 그래프 g에 대한 2차원 배열의 값을 0으로 초기화
            g->adjMatrix[i][j] = 0;
    }
}
```

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프에 정점 삽입

```
// 그래프 g에 정점 v를 삽입하는 연산
void insertVertex(graphType* g, int v)
{
    if (((g->n) + 1) > MAX_VERTEX) {
        printf("Wn 그래프 정점의 개수를 초과하였습니다!");
        return;
    }
    g->n++; // 그래프 정점의 개수 n을 하나 증가
}
```

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프에 간선 삽입

```
// 그래프 g에 간선 (u, v)를 삽입하는 연산
void insertEdge(graphType* g, int u, int v)
{
    // 간선 (u, v)를 삽입하기 위해 정점 u와 v가 그래프에 존재하는지 확인
    if (u >= g->n || v >= g->n) {
        printf("❗n 그래프에 없는 정점입니다!");
        return;
    }
    // 삽입한 간선에 대한 2차원 배열의 원소 값을 1로 설정
    g->adjMatrix[u][v] = 1;
}
```

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프를 배열로 출력

```
// 그래프 g의 2차원 배열 값을 순서대로 출력하는 연산
void print_adjMatrix(graphType* g)
{
    int i, j;
    for (i = 0; i < (g->n); i++) {
        printf("WnWtWt");
        for (j = 0; j < (g->n); j++)
            printf("%2d", g->adjMatrix[i][j]);
    }
}
```

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 공백 그래프 생성

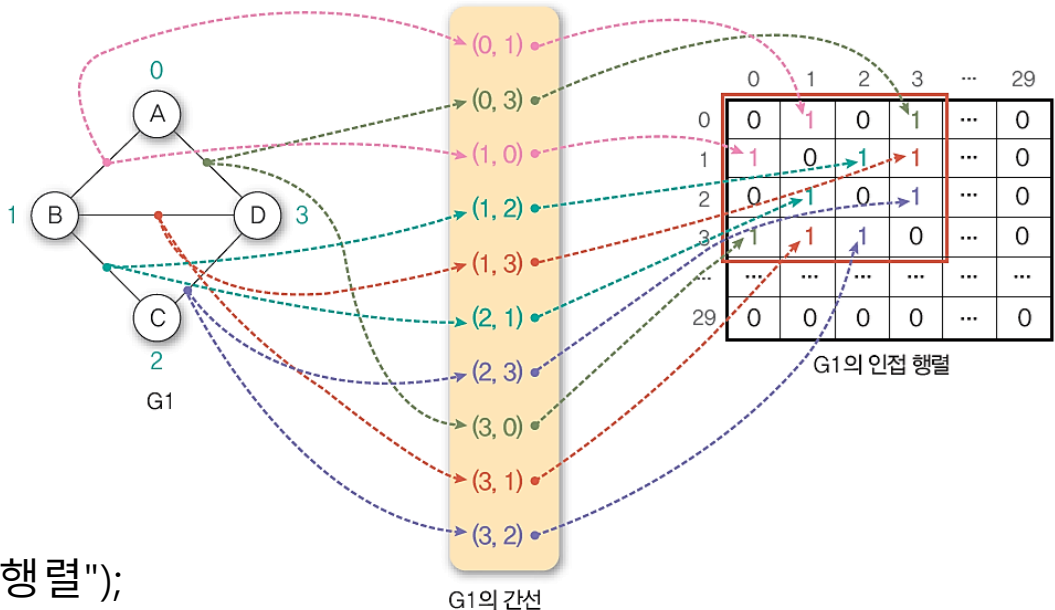
```
void main()
{
    int i;
    graphType *G1, *G2, *G3, *G4;
    G1 = (graphType *)malloc(sizeof(graphType));
    G2 = (graphType *)malloc(sizeof(graphType));
    G3 = (graphType *)malloc(sizeof(graphType));
    G4 = (graphType *)malloc(sizeof(graphType));
    createGraph(G1);
    createGraph(G2);
    createGraph(G3);
    createGraph(G4);
}
```

|     | 0   | 1   | 2   | 3   | ... | 29  |
|-----|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   | 0   | ... | 0   |
| 1   | 0   | 0   | 0   | 0   | ... | 0   |
| 2   | 0   | 0   | 0   | 0   | ... | 0   |
| 3   | 0   | 0   | 0   | 0   | ... | 0   |
| ... | ... | ... | ... | ... | ... | ... |
| 29  | 0   | 0   | 0   | 0   | ... | 0   |

## ❖ 그래프의 인접행렬 표현하기 프로그램

- 그래프 G1의 인접행렬 생성

```
// 그래프 G1
for (i = 0; i<4; i++)
    insertVertex(G1, i); // G1의 정점 0~3 삽입
insertEdge(G1, 0, 1);
insertEdge(G1, 0, 3);
insertEdge(G1, 1, 0);
insertEdge(G1, 1, 2);
insertEdge(G1, 1, 3);
insertEdge(G1, 2, 1);
insertEdge(G1, 2, 3);
insertEdge(G1, 3, 0);
insertEdge(G1, 3, 1);
insertEdge(G1, 3, 2);
printf("G1의 인접 행렬");
print_adjMatrix(G1);
```

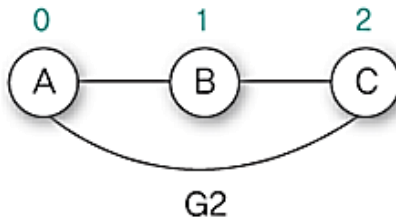




## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프의 G2의 인접행렬 생성

```
// 그래프 G2
for (i = 0; i < 3; i++)
    insertVertex(G2, i); // G2의 정점 0~2 삽입
insertEdge(G2, 0, 1);
insertEdge(G2, 0, 2);
insertEdge(G2, 1, 0);
insertEdge(G2, 1, 2);
insertEdge(G2, 2, 0);
insertEdge(G2, 2, 1);
printf("\n\n G2의 인접 행렬");
print_adjMatrix(G2);
```



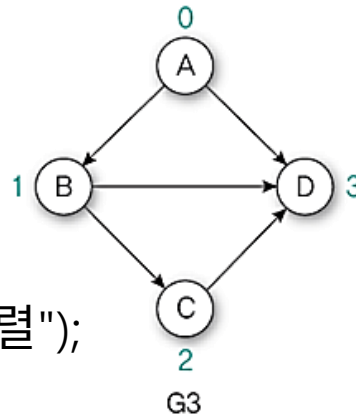
|     | 0   | 1   | 2   | ... | 29  |
|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 1   | 1   | ... | 0   |
| 1   | 1   | 0   | 1   | ... | 0   |
| 2   | 1   | 1   | 0   | ... | 0   |
| ... | ... | ... | ... | ... | 0   |
| 29  | 0   | 0   | 0   | 0   | ... |

G2

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프 G3의 인접행렬 생성

```
// 그래프 G3
for (i = 0; i < 4; i++)
    insertVertex(G3, i); // G3의 정점 0~3 삽입
insertEdge(G3, 0, 1);
insertEdge(G3, 0, 3);
insertEdge(G3, 1, 2);
insertEdge(G3, 1, 3);
insertEdge(G3, 2, 3);
printf("\n\n G3의 인접 행렬");
print_adjMatrix(G3);
```



|     | 0   | 1   | 2   | 3   | ... | 29  |
|-----|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 1   | 0   | 1   | ... | 0   |
| 1   | 0   | 0   | 1   | 1   | ... | 0   |
| 2   | 0   | 0   | 0   | 1   | ... | 0   |
| 3   | 0   | 0   | 0   | 0   | ... | 0   |
| ... | ... | ... | ... | ... | ... | ... |
| 29  | 0   | 0   | 0   | 0   | ... | 0   |

G3

## ❖ 그래프의 인접행렬 표현하기 프로그램

### ■ 그래프의 인접행렬 생성

```
// 그래프 G4
```

```
for (i = 0; i < 3; i++)
```

```
insertVertex(G4, i); // G4의 정점 0~2 삽입
```

```
insertEdge(G4, 0, 1);
```

```
insertEdge(G4, 0, 2);
```

```
insertEdge(G4, 1, 0);
```

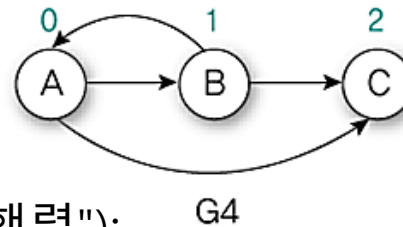
```
insertEdge(G4, 1, 2);
```

```
printf("\n\n G4의 인접 행렬");
```

```
print_adjMatrix(G4);
```

```
getchar();
```

```
}
```

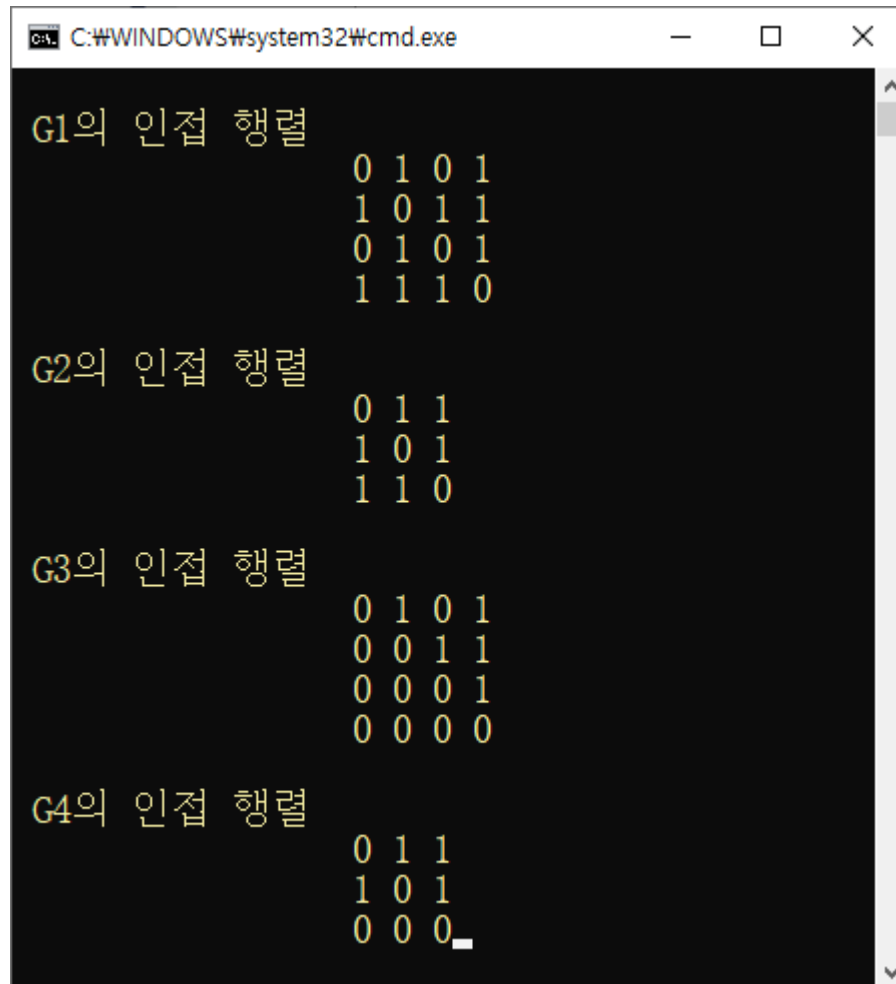


|     | 0   | 1   | 2   | ... | 29  |
|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 1   | 1   | ... | 0   |
| 1   | 1   | 0   | 1   | ... | 0   |
| 2   | 0   | 0   | 0   | ... | 0   |
| ... | ... | ... | ... | ... | 0   |
| 29  | 0   | 0   | 0   | 0   | ... |

G4

## ❖ 그래프의 인접행렬 표현하기 프로그램 : 교재 418p

### ■ 실행 결과



```
C:\WINDOWS\system32\cmd.exe

G1의 인접 행렬
  0 1 0 1
  1 0 1 1
  0 1 0 1
  1 1 1 0

G2의 인접 행렬
  0 1 1
  1 0 1
  1 1 0

G3의 인접 행렬
  0 1 0 1
  0 0 1 1
  0 0 0 1
  0 0 0 0

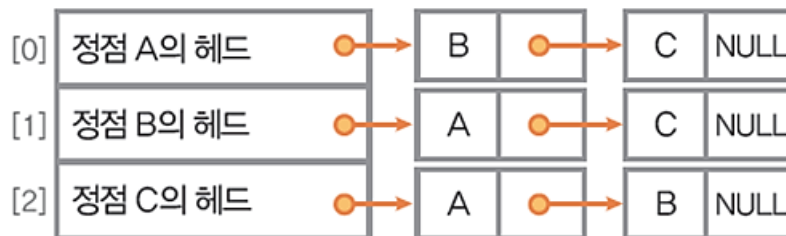
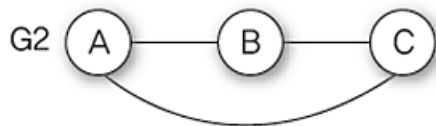
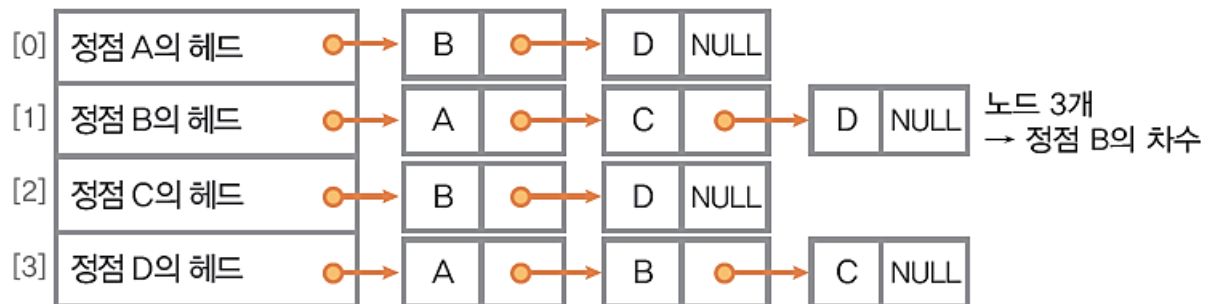
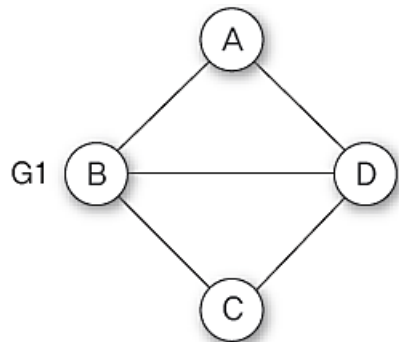
G4의 인접 행렬
  0 1 1
  1 0 1
  0 0 0_
```

## ❖ 연결 자료 구조를 이용한 그래프 표현 : 인접 리스트

- 각 정점에 대한 인접 정점들을 연결하여 만든 단순 연결 리스트
- 각 정점의 차수만큼 노드를 연결
  - 리스트 내의 노드들은 인접 정점에 대해서 오름차순으로 연결
- 인접 리스트의 각 노드
  - 정점을 저장하는 필드와 다음 인접 정점을 연결하는 링크 필드로 구성
  - 정점에 대한 인접 리스트의 헤드포인터는 정점 개수만큼 필요
  - 그래프는 정점의 집합이므로 각 정점에 대한 헤드 포인터를 그룹으로 묶어서 포인터 배열로 구성함

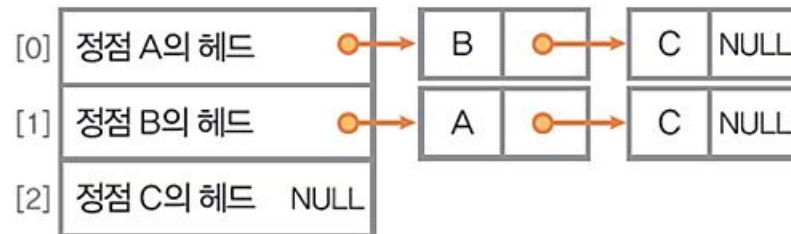
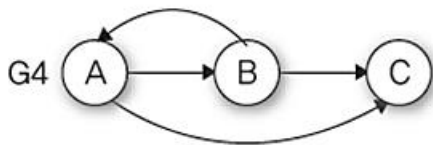
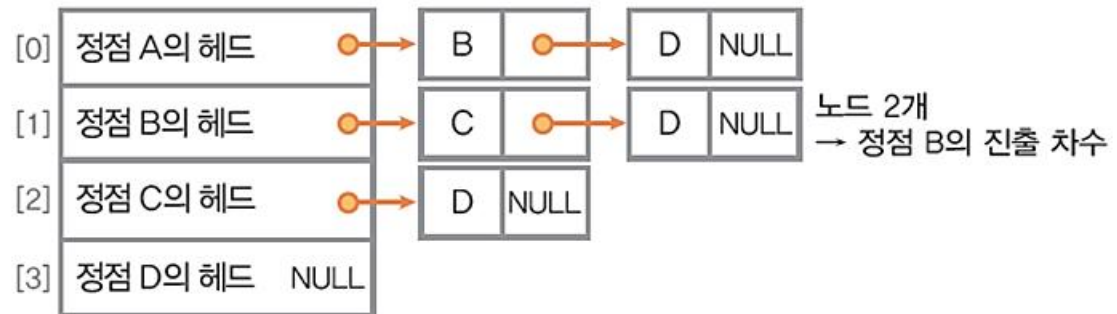
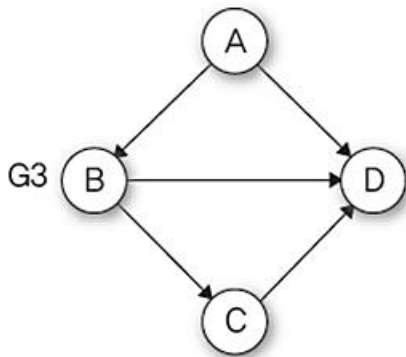
## ❖ 연결 자료 구조를 이용한 그래프 표현 : 인접 리스트

- n개의 정점과 e개의 간선을 가진 **무방향 그래프**의 인접 리스트
  - 헤드 노드 배열의 크기 : n
  - 정점의 모든 차수의 합 =  $2 \times e$
  - 각 정점의 헤드에 연결된 노드의 수 : 정점의 차수



## ❖ 연결 자료 구조를 이용한 그래프 표현 : 인접 리스트

- $n$ 개의 정점과  $e$ 개의 간선을 가진 **방향 그래프**의 인접 리스트
  - 헤드 노드 배열의 크기 :  $n$
  - 정점의 진출 차수의 합 =  $e$
  - 각 정점의 헤드에 연결된 노드의 수 : 정점의 진출 차수



## ❖ 그래프를 인접리스트로 표현하기 프로그램

### ■ 인접 리스트의 노드 구조 정의

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTEX 30 // 헤드 포인터 배열의 최대 크기

// 인접 리스트의 노드 구조를 구조체로 정의
typedef struct graphNode {
    int vertex;           // 정점을 나타내는 데이터 필드
    struct graphNode* link; // 다음 인접 정점을 연결하는 링크 필드
} graphNode;

// 그래프를 인접 리스트로 표현하기 위한 구조체 정의
typedef struct graphType {
    int n; // 그래프의 정점 개수
    graphNode* adjList_H[MAX_VERTEX]; // 그래프 정점에 대한 헤드 포인터 배열
} graphType;
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

### ■ 공백 그래프 생성과 그래프에 정점을 삽입

```
// 공백 그래프를 생성하는 연산
void createGraph(graphType* g)
{
    int v;
    g->n = 0;      // 그래프의 정점 개수를 0으로 초기화
    for (v = 0; v<MAX_VERTEX; v++)
        // 그래프의 정점에 대한 헤드 포인터 배열을 NULL로 초기화
        g->adjList_H[v] = NULL;
}

// 그래프 g에 정점 v를 삽입하는 연산
void insertVertex(graphType* g, int v)
{
    if (((g->n) + 1)>MAX_VERTEX) {
        printf("\n\n 그래프 정점의 개수를 초과하였습니다!");
        return;
    }
    g->n++;        // 그래프의 정점 개수 n을 하나 증가
}
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

### ■ 그래프에 간선을 삽입

```
// 그래프 g에 간선 (u, v)를 삽입하는 연산
void insertEdge(graphType* g, int u, int v)
{
    graphNode* node;

    // 간선 (u, v)를 삽입하기 위해 정점 u와 정점 v가 현재 그래프에 있는지 확인
    if (u >= g->n || v >= g->n) {
        printf("\n\n 그래프에 없는 정점입니다!");
        return;
    }
    node = (graphNode *)malloc(sizeof(graphNode));
    node->vertex = v;
    // 삽입 간선에 대한 노드를 리스트의 첫 번째 노드로 연결
    node->link = g->adjList_H[u];
    g->adjList_H[u] = node;
}
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

### ■ 그래프 정점에 대한 인접 리스트 출력

```
// 그래프 g의 각 정점에 대한 인접 리스트를 출력하는 연산
void print_adjList(graphType* g)
{
    int i;
    graphNode* p;
    for (i = 0; i < g->n; i++) {
        printf("\n\n정점 %c의 인접 리스트", i + 65);
        p = g->adjList_H[i];
        while (p) {
            printf(" -> %c", p->vertex + 65); // 정점 0~3을 A~D로 출력
            p = p->link;
        }
    }
}
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

```
void main()
{
    int i;
    graphType *G1, *G2, *G3, *G4;
    G1 = (graphType *)malloc(sizeof(graphType));
    G2 = (graphType *)malloc(sizeof(graphType));
    G3 = (graphType *)malloc(sizeof(graphType));
    G4 = (graphType *)malloc(sizeof(graphType));
    createGraph(G1);
    createGraph(G2);
    createGraph(G3);
    createGraph(G4);
}
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

```
// 그래프 G1
for (i = 0; i < 4; i++)
    insertVertex(G1, i); // G1의 정점 0~3 삽입
insertEdge(G1, 0, 3);
insertEdge(G1, 0, 1);
insertEdge(G1, 1, 3);
insertEdge(G1, 1, 2);
insertEdge(G1, 1, 0);
insertEdge(G1, 2, 3);
insertEdge(G1, 2, 1);
insertEdge(G1, 3, 2);
insertEdge(G1, 3, 1);
insertEdge(G1, 3, 0);
printf("\n G1의 인접 리스트");
print_adjList(G1);
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

```
// 그래프 G2
for (i = 0; i < 3; i++)
    insertVertex(G2, i); // G2의 정점 0~2 삽입
insertEdge(G2, 0, 2);
insertEdge(G2, 0, 1);
insertEdge(G2, 1, 2);
insertEdge(G2, 1, 0);
insertEdge(G2, 2, 1);
insertEdge(G2, 2, 0);
printf("\n\n G2의 인접 리스트");
print_adjList(G2);
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

```
// 그래프 G3
for (i = 0; i < 4; i++)
    insertVertex(G3, i);    // G3의 정점 0~3 삽입
insertEdge(G3, 0, 3);
insertEdge(G3, 0, 1);
insertEdge(G3, 1, 3);
insertEdge(G3, 1, 2);
insertEdge(G3, 2, 3);
printf("\n\n G3의 인접 리스트");
print_adjList(G3);
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

```
// 그래프 G4
for (i = 0; i < 3; i++)
    insertVertex(G4, i); // G4의 정점 0~2 삽입
insertEdge(G4, 0, 2);
insertEdge(G4, 0, 1);
insertEdge(G4, 1, 2);
insertEdge(G4, 1, 0);
printf("\n\n G4의 인접 리스트");
print_adjList(G4);

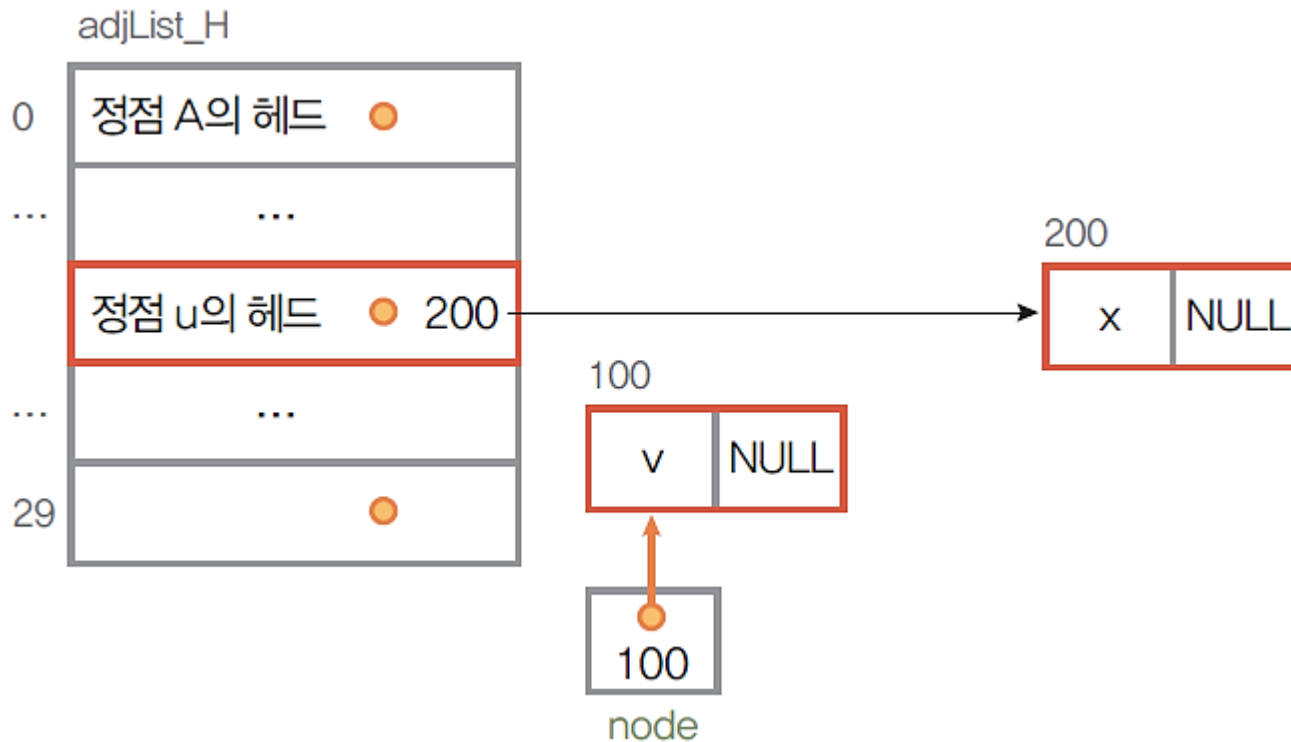
getchar();
}
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

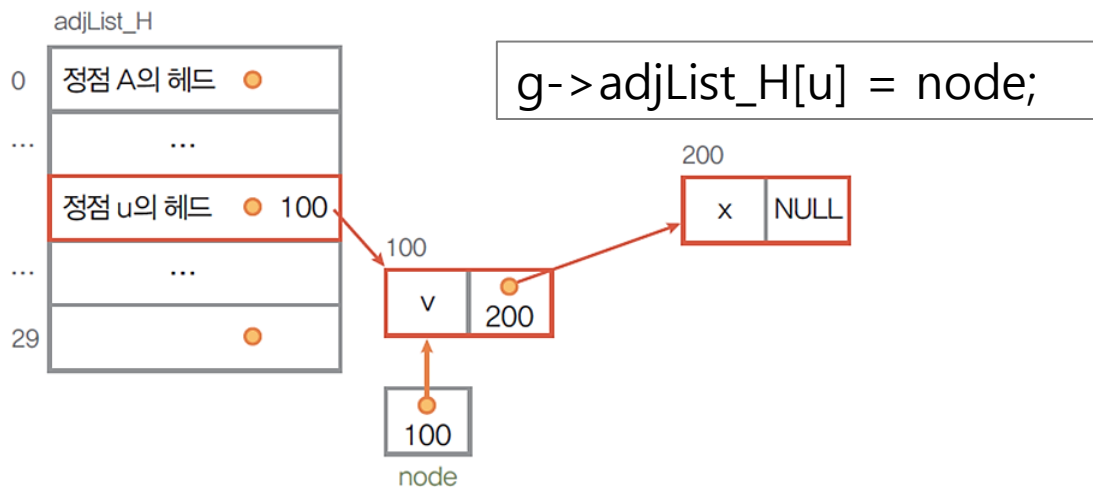
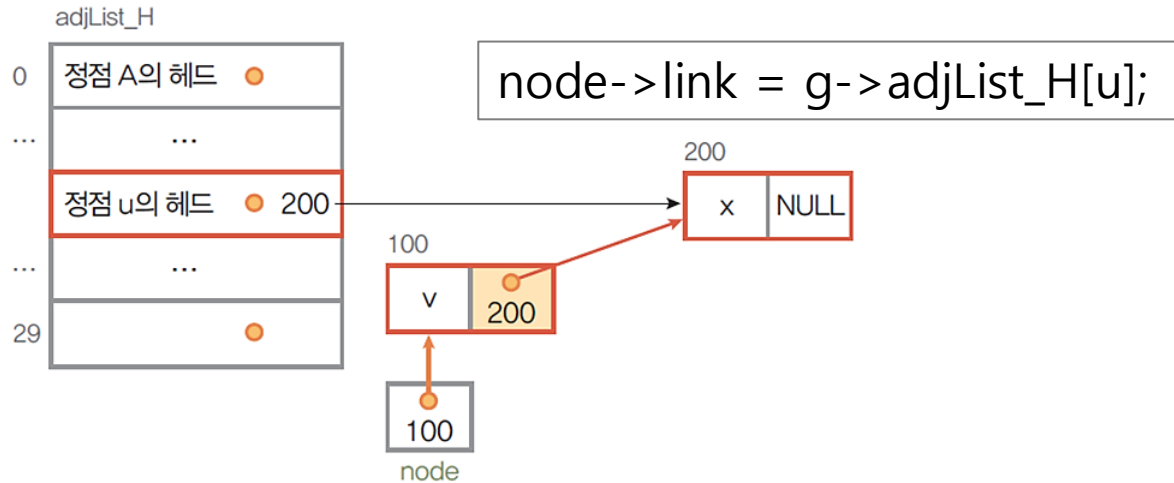
- 그래프  $g$ 에 새로운 간선( $u, v$ )를 삽입하는 연산

```
node->vertex = v;
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 g에 새로운 간선(u, v)를 삽입하는 연산



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1, G2, G3, G4에 대한 메모리를 할당하고, 헤드 포인터 배열이 NULL로 초기화된 공백 그래프를 생성
- 그래프 G1에 정점 네 개와 간선 열 개를 삽입하여 인접 리스트를 구성하고 출력

```
for (i = 0; i < 4; i++)  
    insertVertex(G1, i);
```

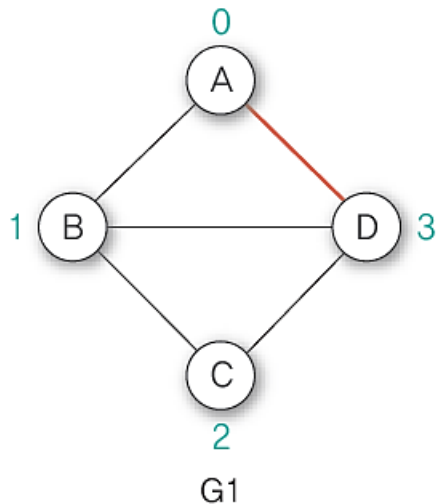
| adjList_H |                 |
|-----------|-----------------|
| 0         | 정점 A의 헤드 ○ NULL |
| 1         | 정점 B의 헤드 ○ NULL |
| 2         | 정점 C의 헤드 ○ NULL |
| 3         | 정점 D의 헤드 ○ NULL |
| ...       | ...             |
| 29        | ○ NULL          |

G1

## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(0, 3)을 삽입

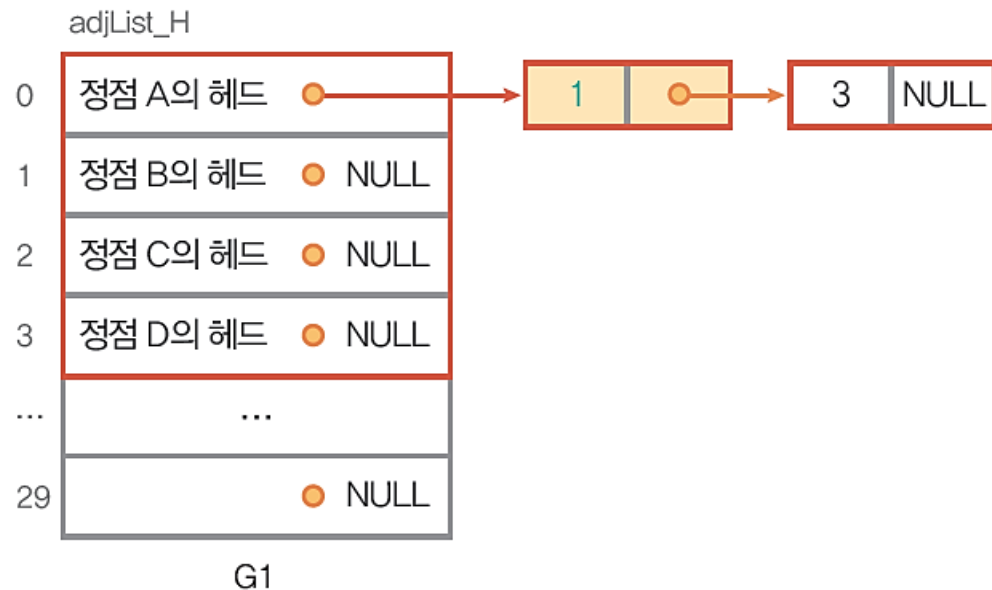
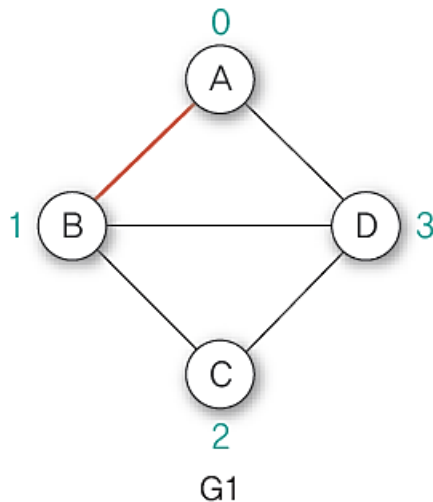
```
insertEdge(G1, 0, 3);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(0, 1)을 삽입

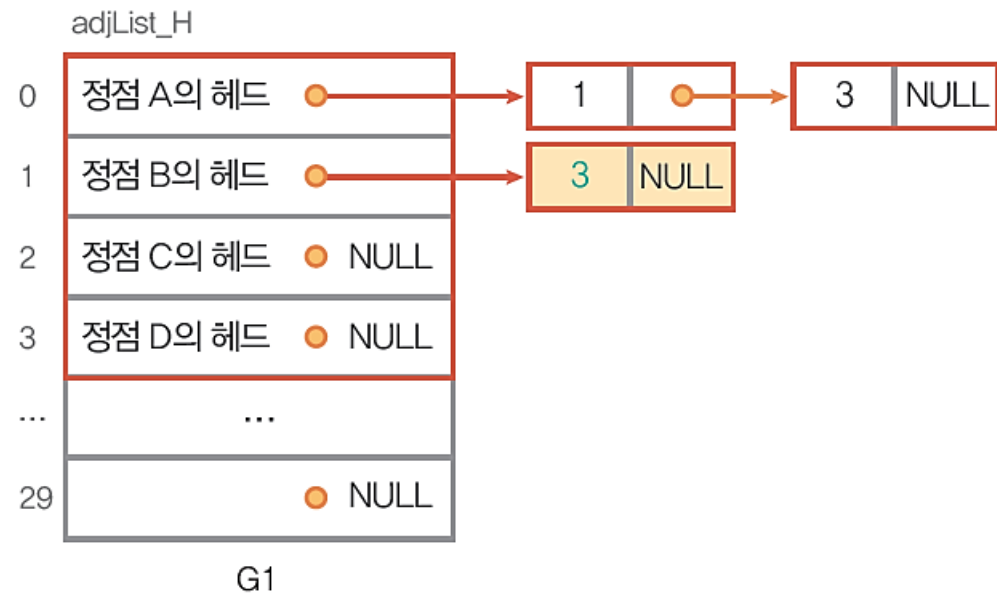
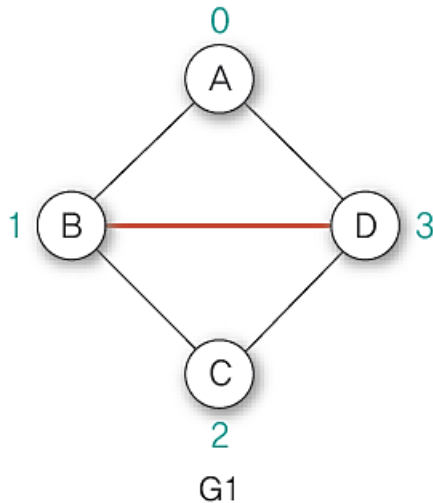
```
insertEdge(G1, 0, 1);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(1, 3)을 삽입

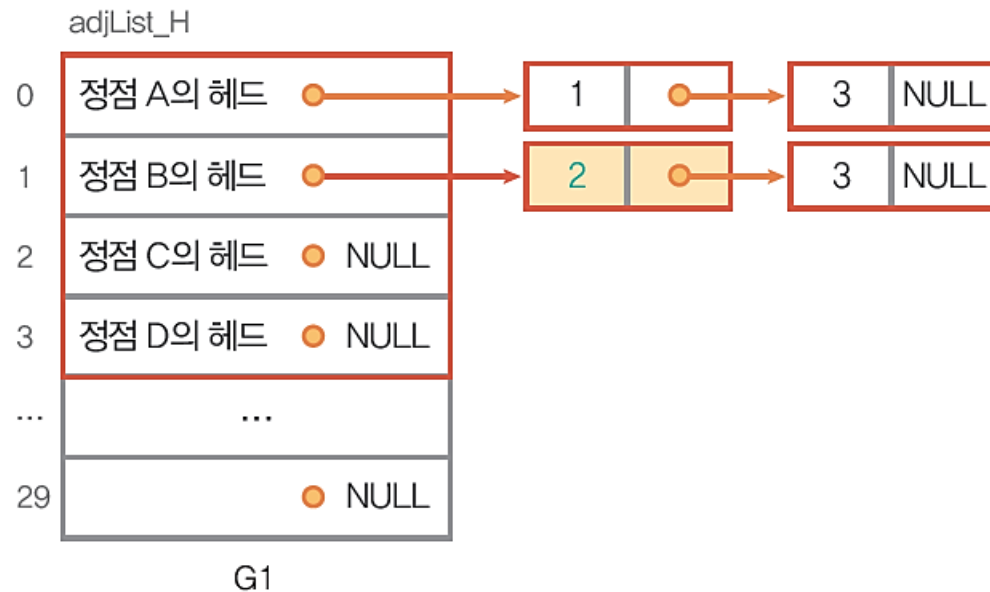
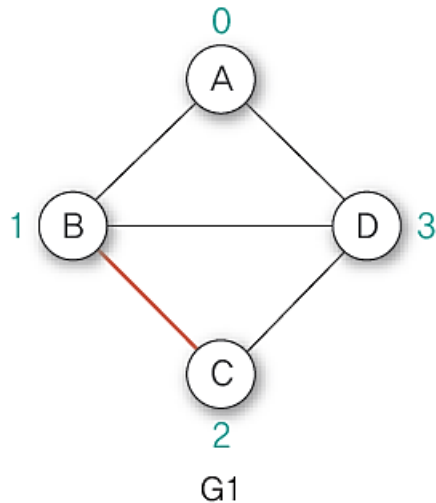
```
insertEdge(G1, 1, 3);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(1, 2)을 삽입

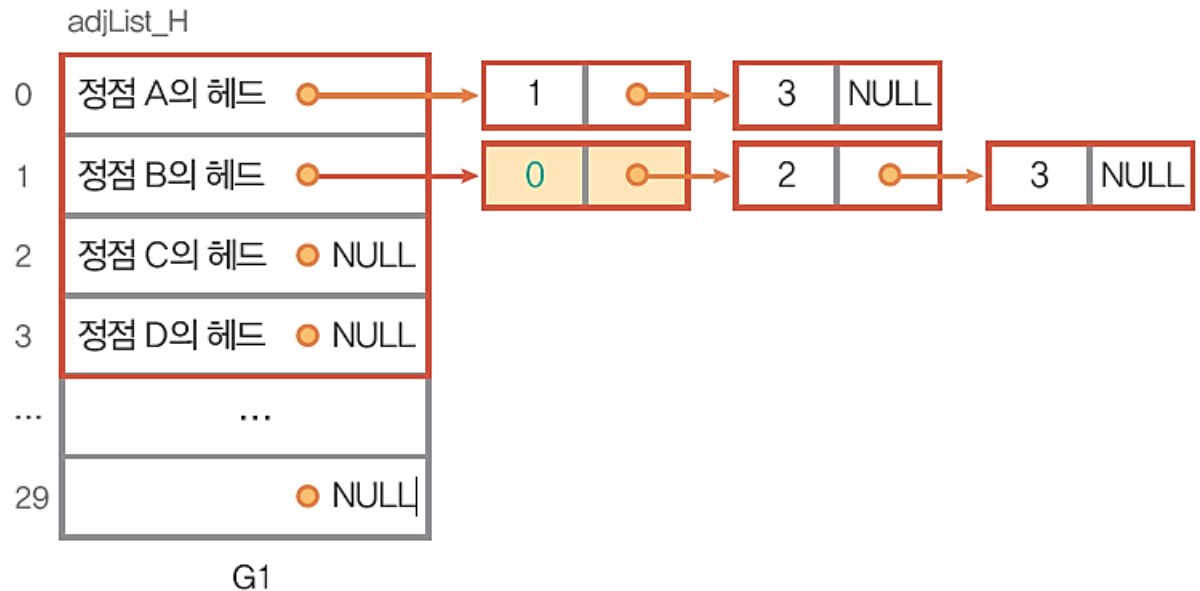
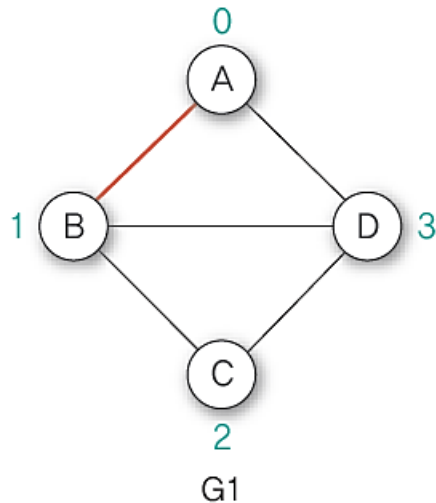
```
insertEdge(G1, 1, 2);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(1, 0)을 삽입

```
insertEdge(G1, 1, 0);
```

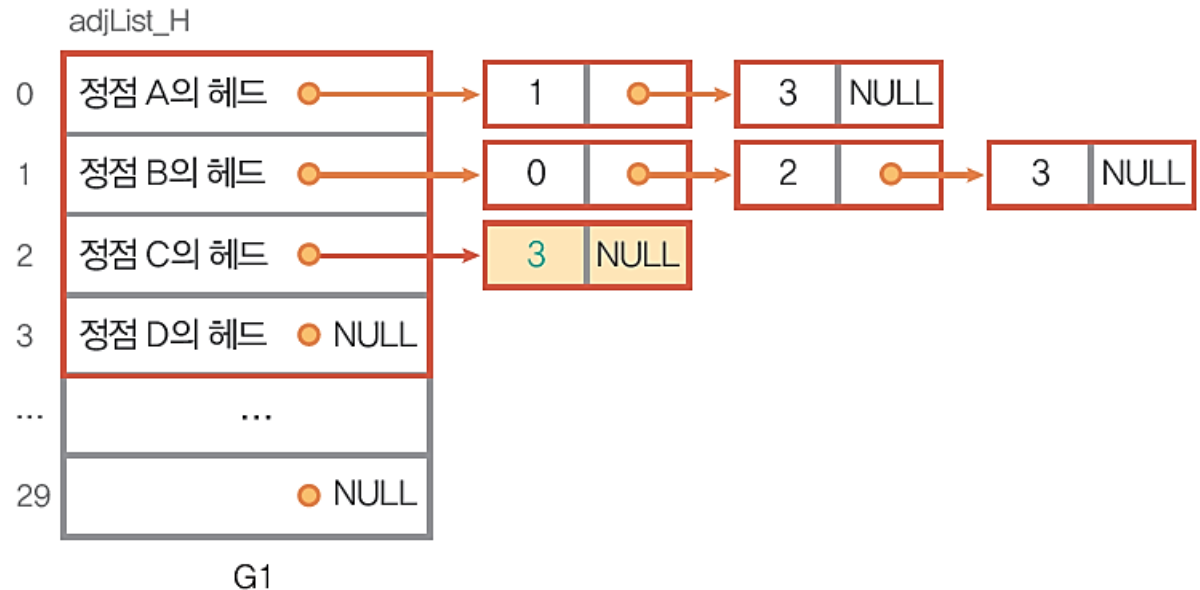
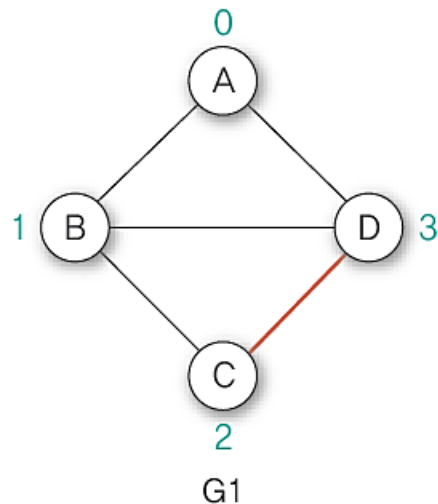




## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(2, 3)을 삽입

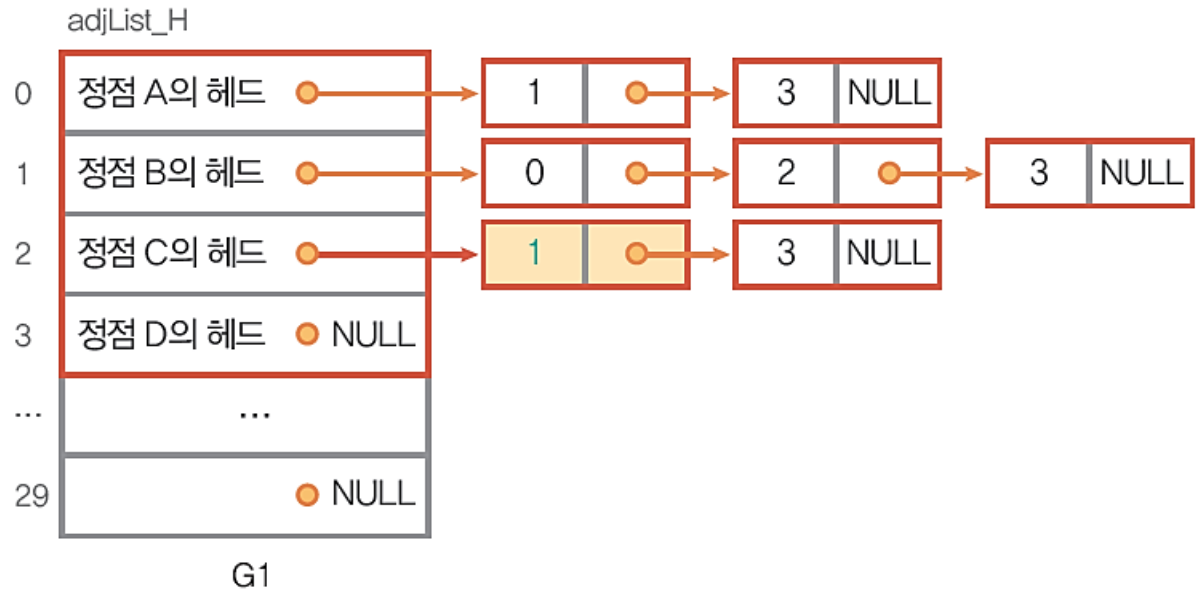
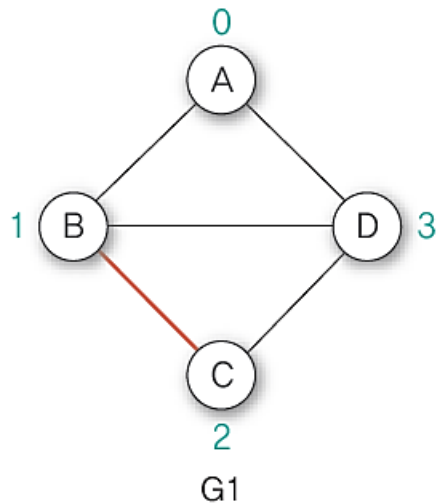
```
insertEdge(G1, 2, 3);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(2, 1)을 삽입

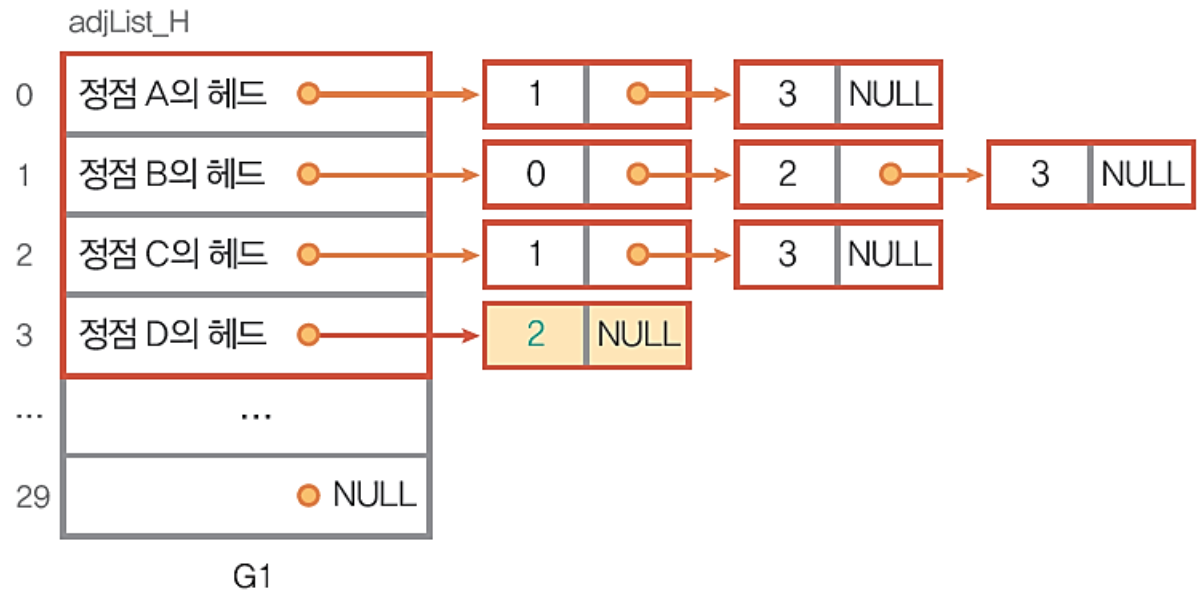
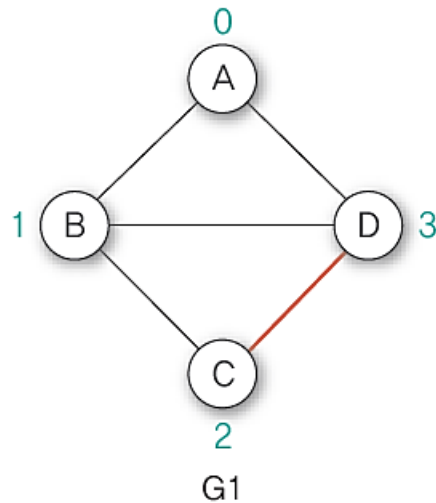
```
insertEdge(G1, 2, 1);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(3, 2)을 삽입

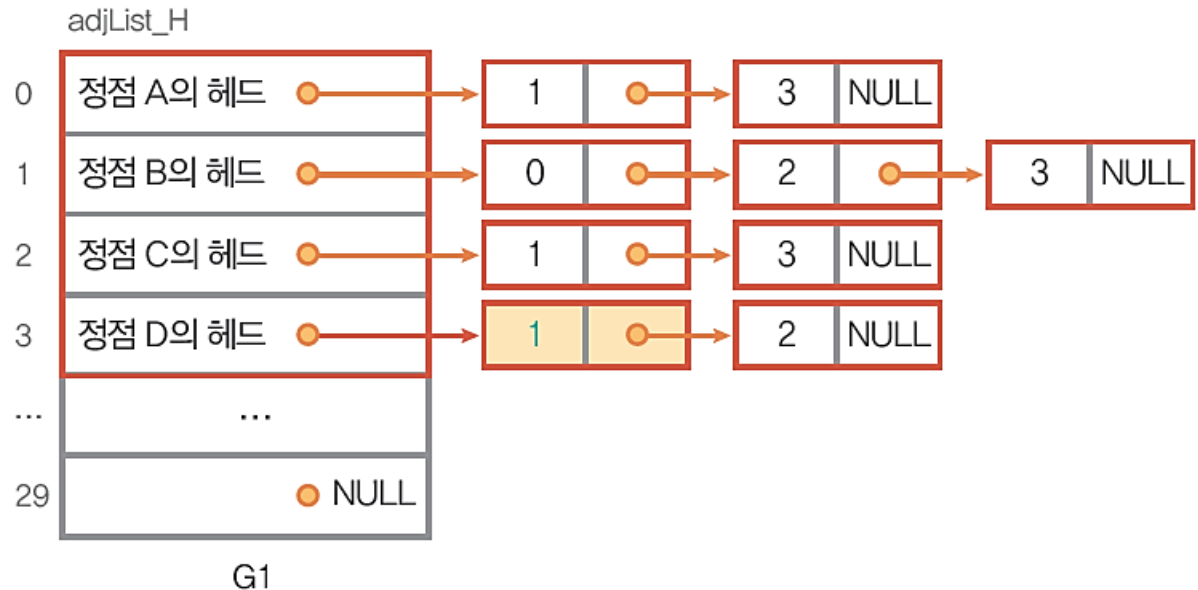
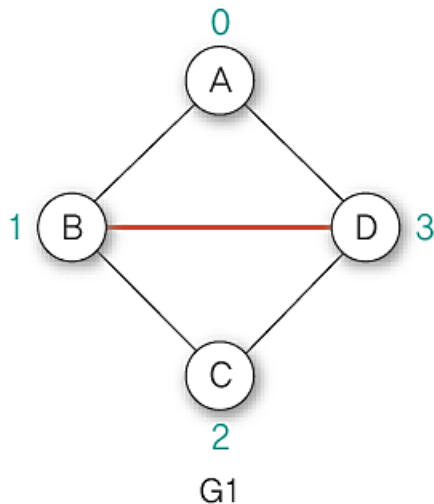
```
insertEdge(G1, 3, 2);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G1에 간선(3, 1)을 삽입

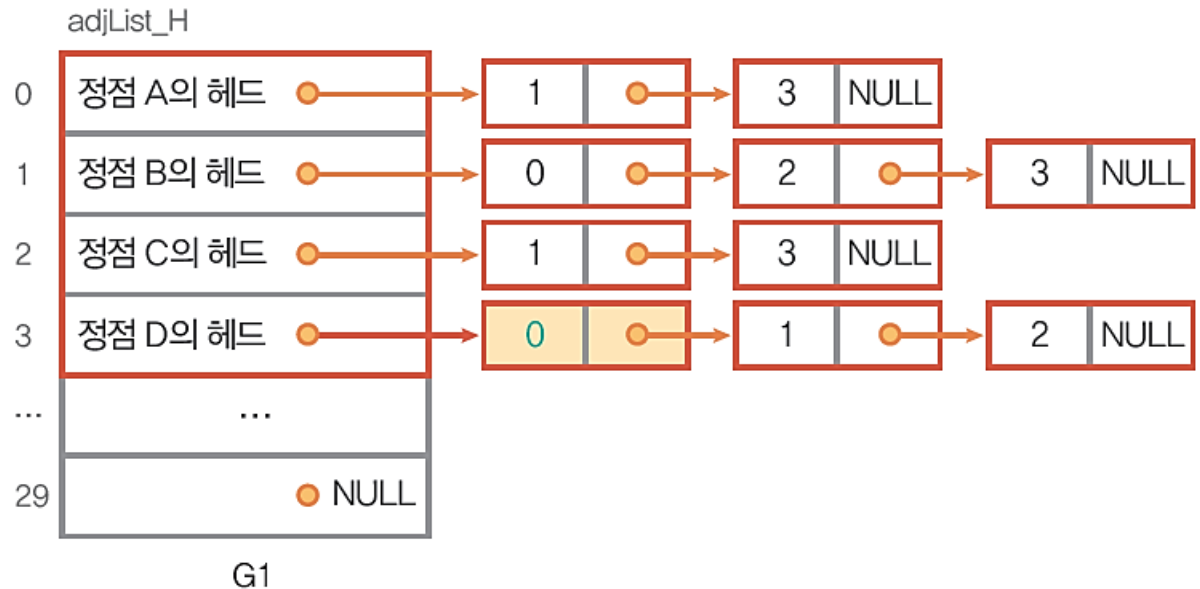
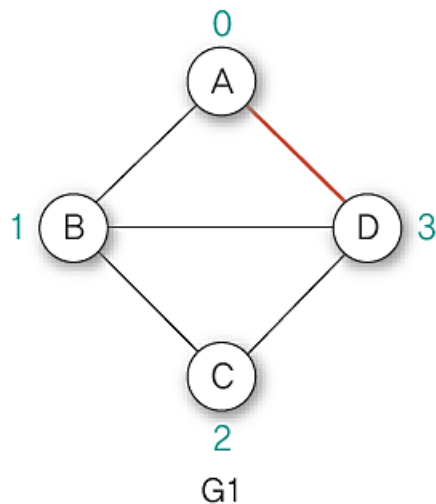
```
insertEdge(G1, 3, 1);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램

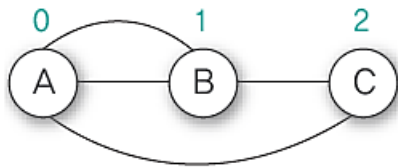
- 그래프 G1에 간선(3, 0)을 삽입

```
insertEdge(G1, 3, 0);
```

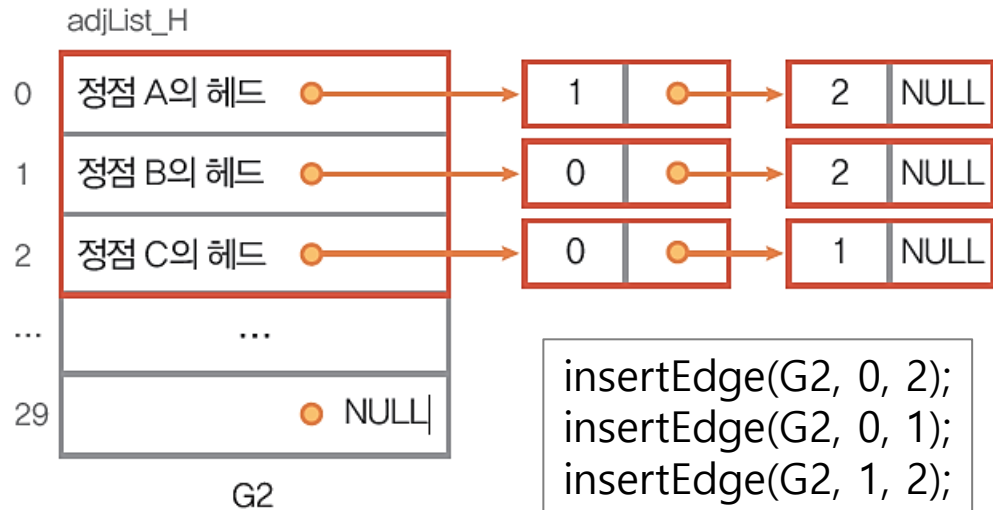


## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G2에 간선 여섯 개 (0, 2), (0, 1), (1, 2), (1,0), (2, 1), (2, 0) 삽입



G2

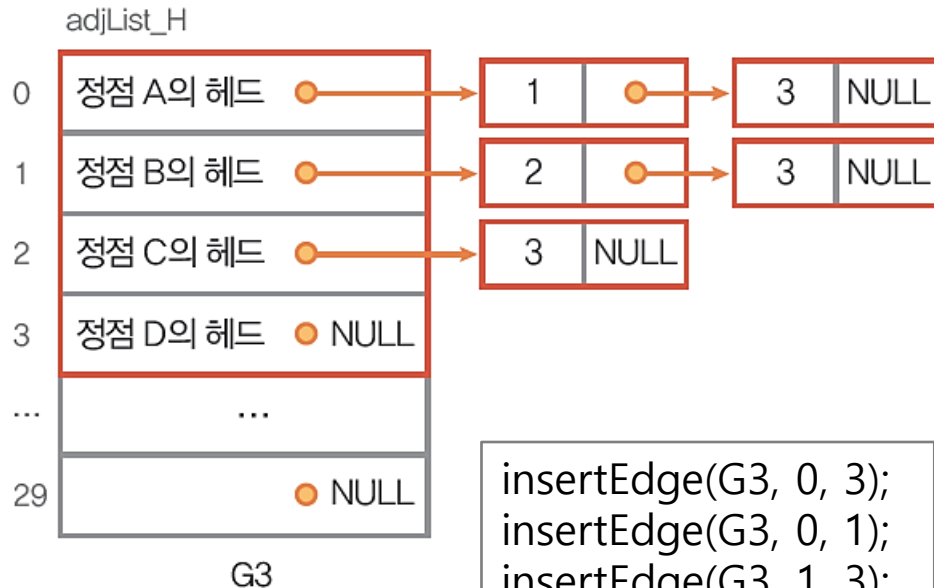
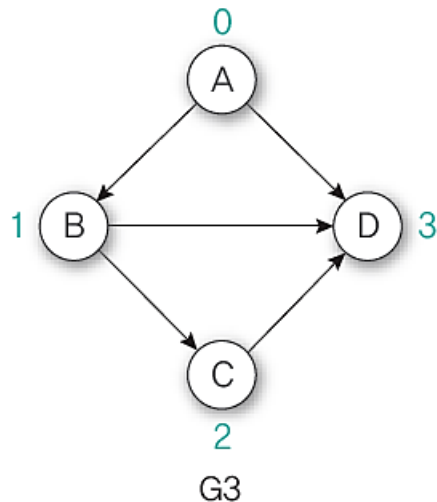


```
for (i = 0; i < 3; i++)  
    insertVertex(G2, i);
```

```
insertEdge(G2, 0, 2);  
insertEdge(G2, 0, 1);  
insertEdge(G2, 1, 2);  
insertEdge(G2, 1, 0);  
insertEdge(G2, 2, 1);  
insertEdge(G2, 2, 0);
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G3에 간선을 다섯 개 ( $\langle 0, 3 \rangle$ ,  $\langle 0, 1 \rangle$ ,  $\langle 1, 3 \rangle$ ,  $\langle 1, 2 \rangle$ ,  $\langle 2, 3 \rangle$ ) 삽입

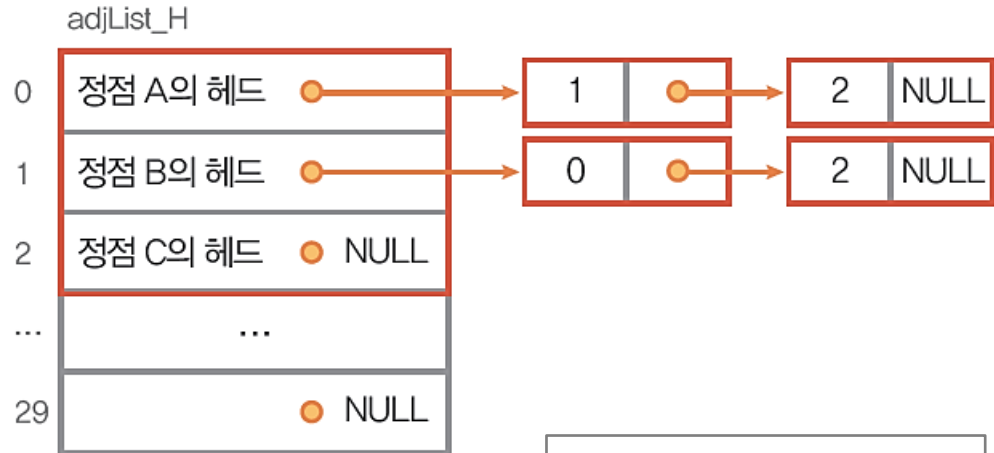
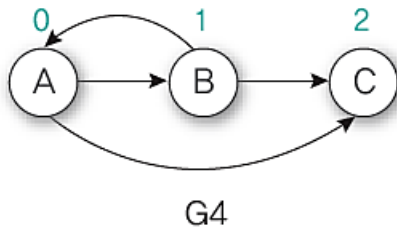


```
for (i = 0; i < 4; i++)  
    insertVertex(G3, i);
```

```
insertEdge(G3, 0, 3);  
insertEdge(G3, 0, 1);  
insertEdge(G3, 1, 3);  
insertEdge(G3, 1, 2);  
insertEdge(G3, 2, 3);
```

## ❖ 그래프를 인접리스트로 표현하기 프로그램

- 그래프 G4에 간선을 네 개(<0, 2>, <0, 1>, <1, 2>, <1, 0>) 삽입



```
for (i = 0; i < 3; i++)  
    insertVertex(G4, i);
```

```
insertEdge(G4, 0, 2);  
insertEdge(G4, 0, 1);  
insertEdge(G4, 1, 2);  
insertEdge(G4, 1, 0);
```



## ❖ 그래프를 인접리스트로 표현하기 프로그램 : [교재 425p](#)

### ■ 실행 결과

```
C:\WINDOWS\system32\cmd.exe

G1의 인접 리스트
정점 A의 인접 리스트 -> B -> D
정점 B의 인접 리스트 -> A -> C -> D
정점 C의 인접 리스트 -> B -> D
정점 D의 인접 리스트 -> A -> B -> C

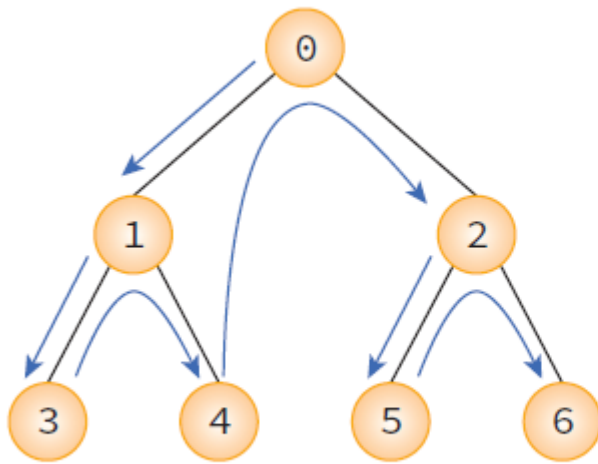
G2의 인접 리스트
정점 A의 인접 리스트 -> B -> C
정점 B의 인접 리스트 -> A -> C
정점 C의 인접 리스트 -> A -> B

G3의 인접 리스트
정점 A의 인접 리스트 -> B -> D
정점 B의 인접 리스트 -> C -> D
정점 C의 인접 리스트 -> D
정점 D의 인접 리스트

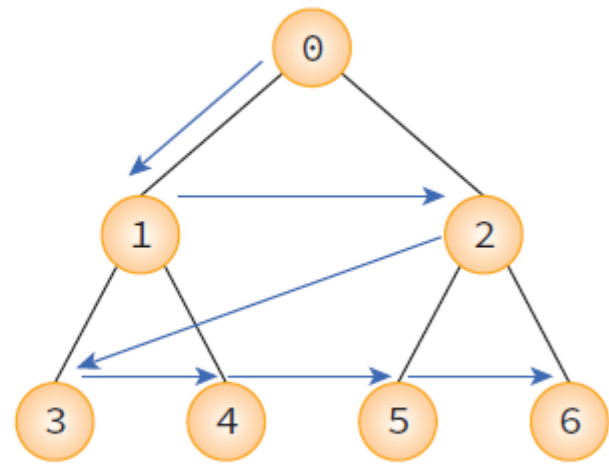
G4의 인접 리스트
정점 A의 인접 리스트 -> B -> C
정점 B의 인접 리스트 -> A -> C
정점 C의 인접 리스트
```

## ❖ 그래프 순회(graph traversal), 그래프 탐색(graph search)

- 하나의 정점에서 시작하여 그래프에 있는 모든 정점을 한번씩 방문하여 처리하는 연산
- 그래프 탐색방법
  - 깊이 우선 탐색(depth first search : DFS)
  - 너비 우선 탐색(breadth first search : BFS)



깊이 우선 탐색



너비 우선 탐색

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

### ■ 순회 방법

- ① 시작 정점의 한 방향으로 갈 수 있는 경로가 있는 곳까지 깊이 탐색
  - ② 더 이상 갈 곳이 없으면, 가장 마지막에 만났던 갈림길 간선이 있는 정점으로 되돌아옴
  - ③ 다른 방향의 간선으로 탐색을 계속 반복하여 결국 모든 정점을 방문
- ※ 가장 마지막에 만났던 갈림길 간선의 정점으로 가장 먼저 되돌아가서 다시 깊이 우선 탐색을 반복해야 하므로 후입 선출 구조의 스택 사용

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

### ■ 수행 순서

- ① 시작 정점  $v$ 를 결정하여 방문한다.
- ② 정점  $v$ 에 인접한 정점 중에서
  - ⓐ 방문하지 않은 정점  $w$ 가 있으면, 정점  $v$ 를 스택에 push하고  $w$ 를 방문한다. 그리고  $w$ 를 다시  $v$ 로 하여 다시 ②를 반복한다.
  - ⓑ 방문하지 않은 정점이 없으면, 탐색의 방향을 바꾸기 위해서 스택을 pop하여 받은 가장 마지막 방문 정점을  $v$ 로 하여 다시 ②를 수행한다.
- ③ 스택이 공백이 될 때까지 ②를 반복한다.

## ❖ 깊이 우선 탐색 알고리즘

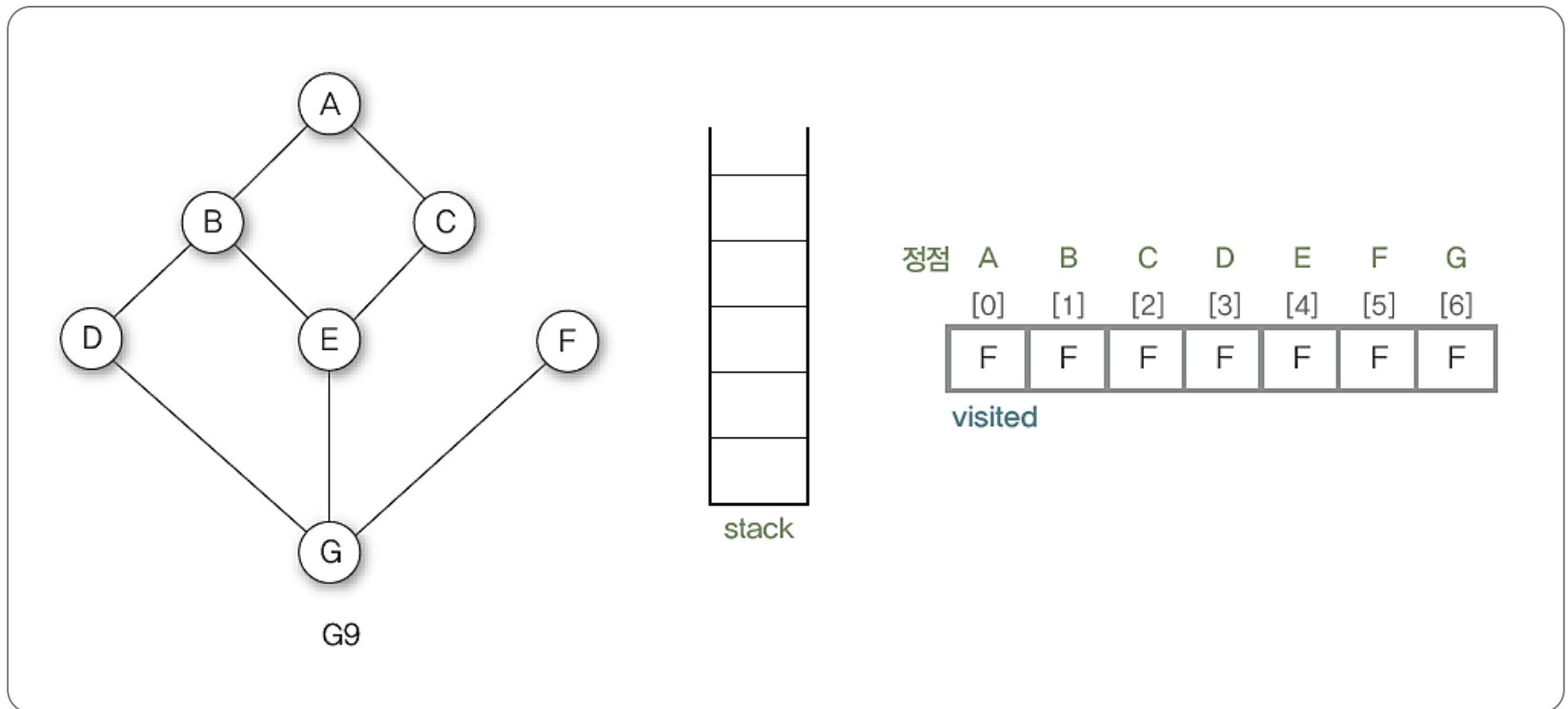
### 알고리즘

### 그래프의 깊이 우선탐색

```
DFS(v)
  for (i ← 0; i < n; i ← i + 1) do {
    visited[i] ← false;
  }
  stack ← createStack();
  visited[v] ← true;
  while (not isEmpty(stack)) do {
    if (visited[v의 인접정점 w] = false) then {
      push(stack, v);
      visited[w] ← true;
      w 방문;
      v ← w;
    }
    else v ← pop(stack);
  }
end DFS()
```

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

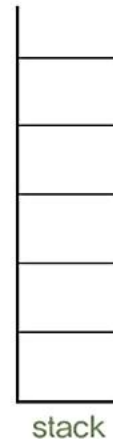
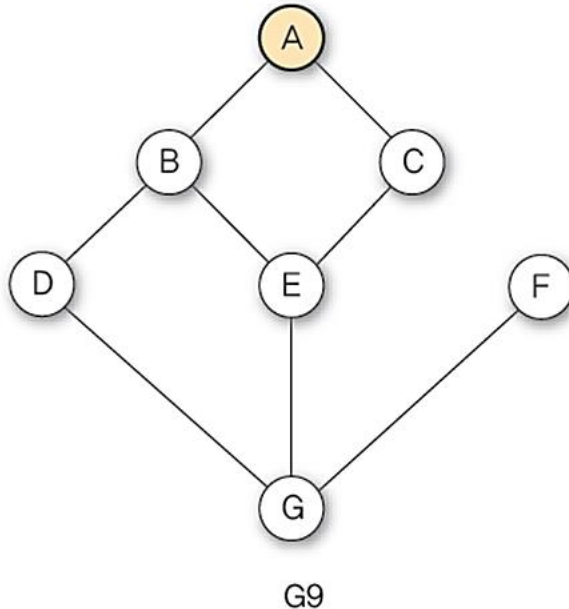
- 예) 그래프 G9에 대한 깊이 우선 탐색
- 초기 상태 : 배열 visited를 False로 초기화 하고, 공백 스택을 생성



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

① 정점 A를 시작으로 깊이 우선 탐색을 시작

```
visited[A] ← true;  
A 방문;
```

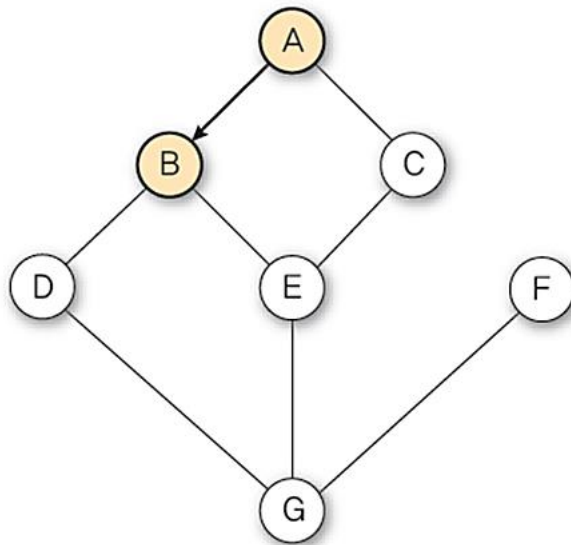


| 정점      | A   | B   | C   | D   | E   | F   | G   |
|---------|-----|-----|-----|-----|-----|-----|-----|
|         | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
| visited | T   | F   | F   | F   | F   | F   | F   |

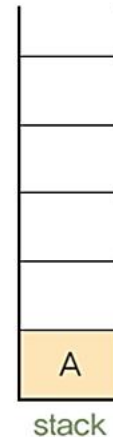
## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ② 정점 A에 방문하지 않은 정점 B, C가 있으므로 A를 스택에 push하고 인접 정점 B와 C 중에서 오름차순에 따라 B를 선택하여 탐색을 계속 함

```
push(stack, A);  
visited[B] ← true;  
B 방문;
```



G9



A의 인접 정점

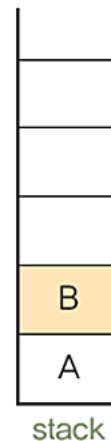
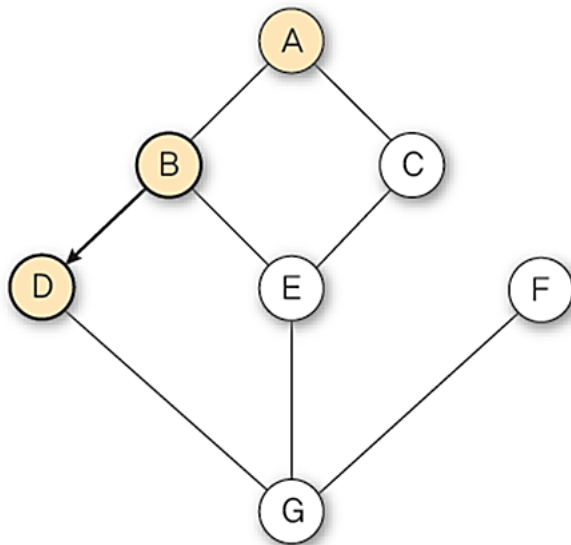
|         |     |     |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|-----|-----|
| 정점      | A   | B   | C   | D   | E   | F   | G   |
|         | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
|         | T   | T   | F   | F   | F   | F   | F   |
| visited |     |     |     |     |     |     |     |



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ③ 정점 B에 방문하지 않은 정점 D, E가 있으므로 B를 스택에 push하고, 방문하지 않은 인접 정점 D와 E 중에서 오름차순에 따라 D를 선택하여 탐색을 계속

```
push(stack, B);  
visited[D] ← true;  
D 방문;
```



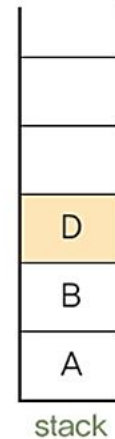
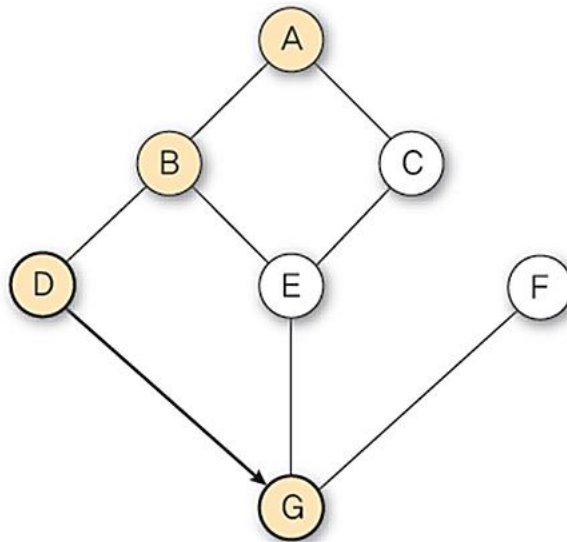
B의 인접 정점

|         |     |     |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|-----|-----|
| 정점      | A   | B   | C   | D   | E   | F   | G   |
|         | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
|         | T   | T   | F   | T   | F   | F   | F   |
| visited |     |     |     |     |     |     |     |

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ④ 정점 D에 방문하지 않은 정점 G가 있으므로 D를 스택에 push하고, 인접정점 G를 선택하여 탐색을 계속한다.

```
push(stack, D);  
visited[G] ← true;  
G 방문;
```



D의 인접 정점

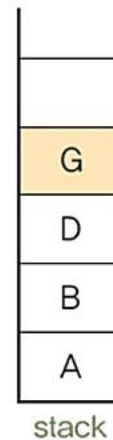
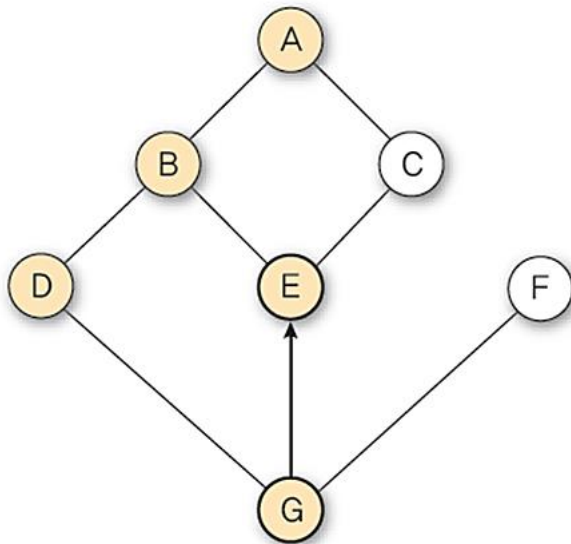
|    |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|
| 정점 | A   | B   | C   | D   | E   | F   | G   |
|    | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
|    | T   | T   | F   | T   | F   | F   | T   |

visited

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

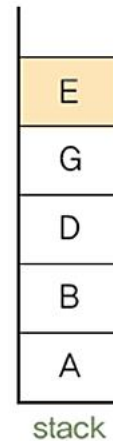
- ⑤ 정점 G에 방문하지 않은 정점 E, F가 있으므로 G를 스택에 push 하고, 인접정점 E와 F 중에서 오름차순에 따라 E를 선택하여 탐색을 계속한다.

```
push(stack, G);  
visited[E] ← true;  
E 방문;
```



|         |     |     |     |          |     |     |     |  |
|---------|-----|-----|-----|----------|-----|-----|-----|--|
|         |     |     |     | G의 인접 정점 |     |     |     |  |
| 정점      | A   | B   | C   | D        | E   | F   | G   |  |
|         | [0] | [1] | [2] | [3]      | [4] | [5] | [6] |  |
|         | T   | T   | F   | T        | T   | F   | T   |  |
| visited |     |     |     |          |     |     |     |  |

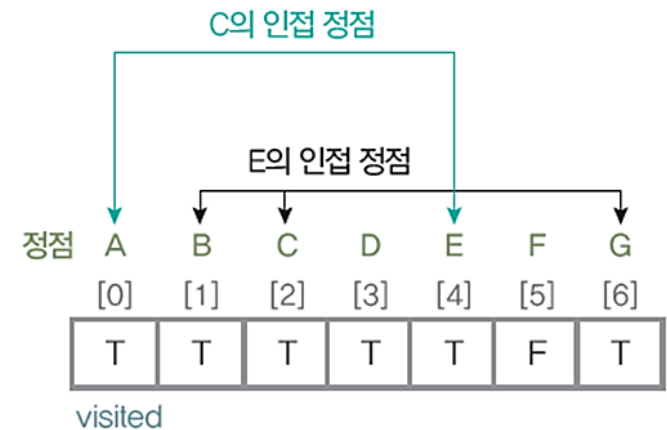
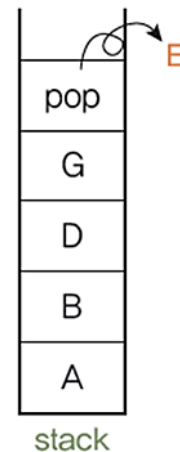
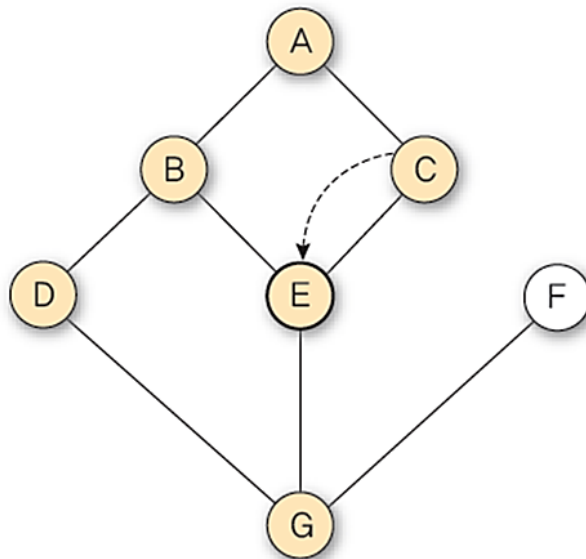
- ```
push(stack, E);
visited[C] ← true;
C 방문;
```



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑦ 정점 C에서 방문하지 않은 인접정점이 없으므로, 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 E에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

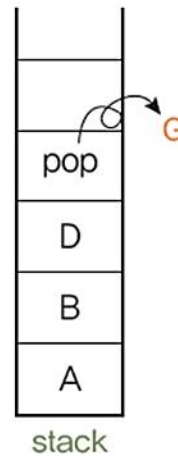
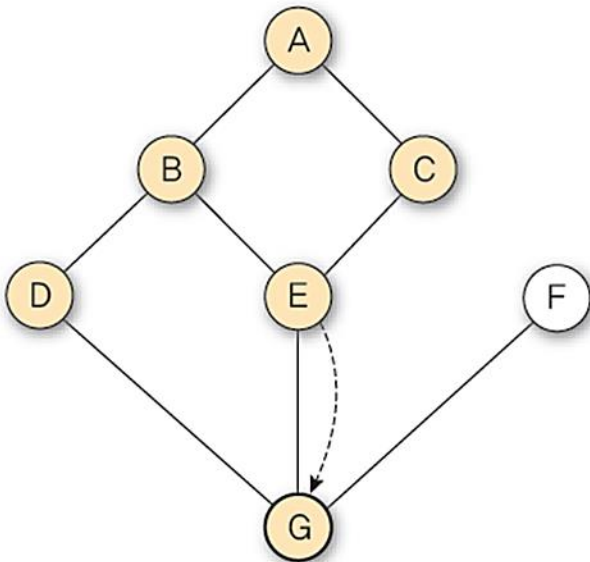
```
pop(stack);
```



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑧ 정점 E는 방문하지 않은 인접정점이 없으므로, 다시 스택을 pop 하여 받은 정점 G에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

```
pop(stack);
```

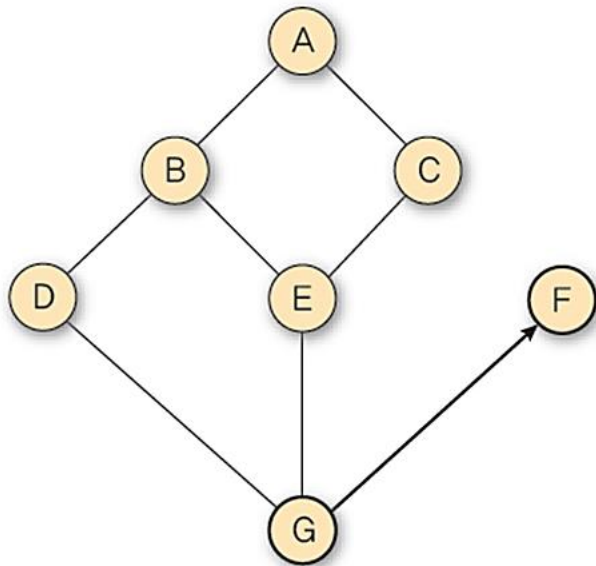


| 정점      | A   | B   | C   | D   | E   | F   | G   |
|---------|-----|-----|-----|-----|-----|-----|-----|
|         | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
| visited | T   | T   | T   | T   | T   | F   | T   |

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑨ 현재 정점 G에서 방문하지 않은 정점 F가 있으므로 G를 스택에 push하고, 인접 정점 F를 선택하여 탐색을 계속한다.

```
push(stack, G);  
visited[F] ← true;  
F 방문;
```

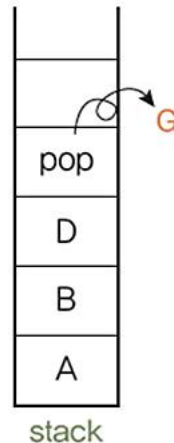
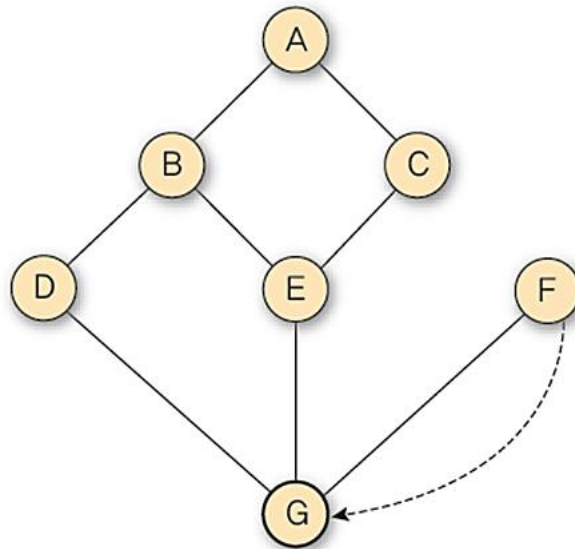


|         |     |     |          |     |     |     |
|---------|-----|-----|----------|-----|-----|-----|
|         |     |     |          |     |     |     |
|         |     |     | G의 인접 정점 |     |     |     |
| 정점      | A   | B   | C        | D   | E   | F   |
|         | [0] | [1] | [2]      | [3] | [4] | [5] |
|         | T   | T   | T        | T   | T   | T   |
| visited |     |     |          |     |     |     |

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑩ 정점 F에서 방문하지 않은 인접정점이 없으므로, 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 G에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

```
pop(stack);
```



G의 인접 정점

|    |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|
| 정점 | A   | B   | C   | D   | E   | F   | G   |
|    | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
|    | T   | T   | T   | T   | T   | T   | T   |

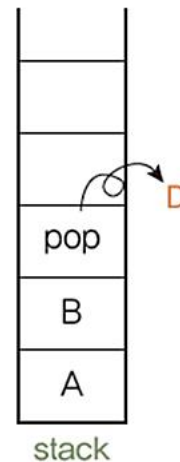
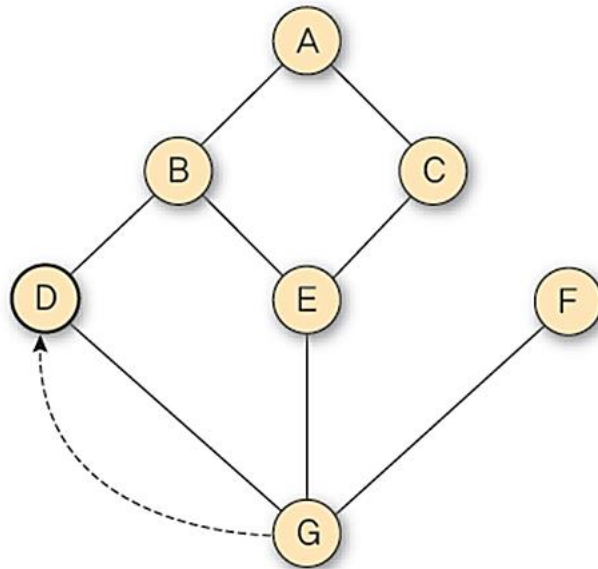
visited



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑪ 정점 G에서 방문하지 않은 인접정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 D에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

```
pop(stack);
```

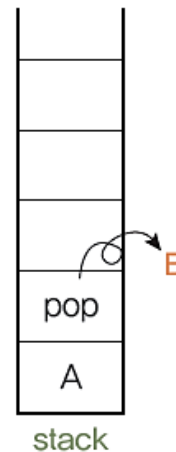
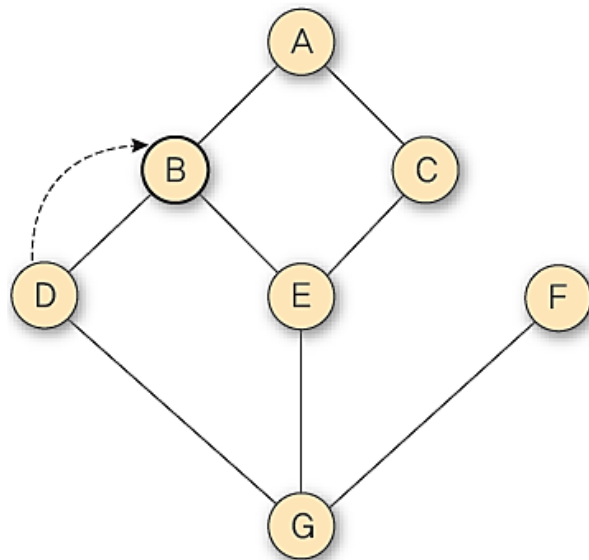


|         |     |          |     |     |     |     |     |
|---------|-----|----------|-----|-----|-----|-----|-----|
|         |     | D의 인접 정점 |     |     |     |     |     |
| 정점      | A   | B        | C   | D   | E   | F   | G   |
|         | [0] | [1]      | [2] | [3] | [4] | [5] | [6] |
|         | T   | T        | T   | T   | T   | T   | T   |
| visited |     |          |     |     |     |     |     |

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑫ 정점 D에서 방문하지 않은 인접정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 B에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

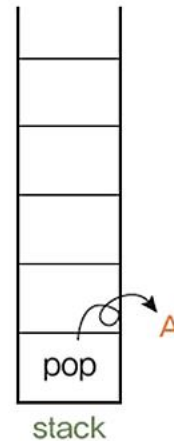
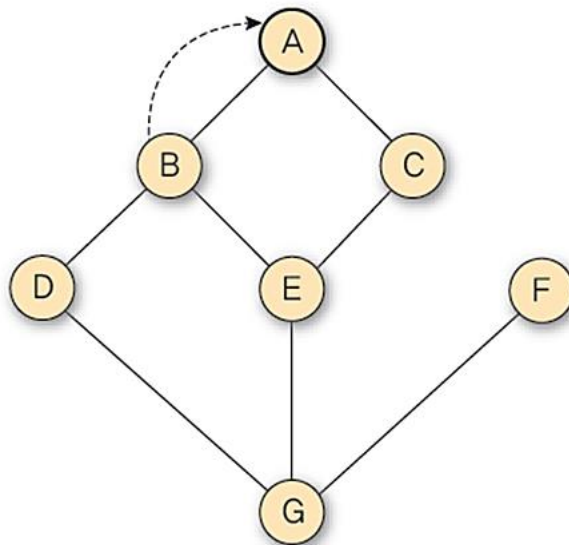
```
pop(stack);
```



## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑬ 정점 B에서 방문하지 않은 인접정점이 없으므로, 다시 마지막 정점으로 돌아가기 위해 스택을 pop 하여 받은 정점 A에 대해서 방문하지 않은 인접정점이 있는지 확인한다.

```
pop(stack);
```

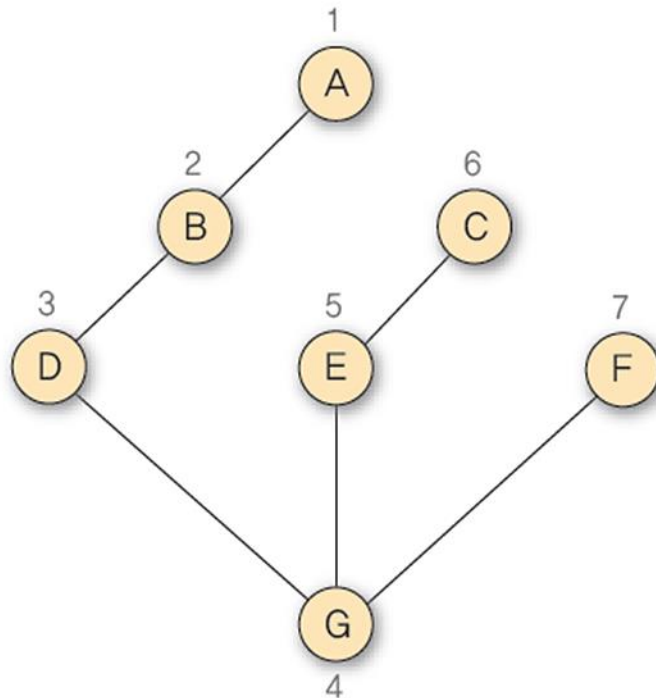


A의 인접 정점

|         |     |     |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|-----|-----|
| 정점      | A   | B   | C   | D   | E   | F   | G   |
|         | [0] | [1] | [2] | [3] | [4] | [5] | [6] |
|         | T   | T   | T   | T   | T   | T   | T   |
| visited |     |     |     |     |     |     |     |

## ❖ 깊이 우선 탐색(Depth First Search : DFS)

- ⑭ 정점 A에서 방문하지 않은 인접정점이 없으므로, 마지막 정점으로 돌아가기 위해 스택을 pop 하는데 스택이 공백이므로 깊이 우선 탐색을 종료한다.
- 그래프 G9의 깊이 우선 탐색 경로 : A-B-D-G-E-C-F



## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTEX 10
#define FALSE 0
#define TRUE 1

// 그래프에 대한 인접 리스트의 노드 구조 정의
typedef struct graphNode {
    int vertex;
    struct graphNode* link;
} graphNode;

typedef struct graphType {
    int n; // 정점 개수
    // 정점에 대한 인접 리스트의 헤드 노드 배열
    graphNode* adjList_H[MAX_VERTEX];
    int visited[MAX_VERTEX]; // 정점에 대한 방문 표시를 위한 배열
} graphType;
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
typedef int element;

typedef struct stackNode {
    int data;
    struct stackNode *link;
} stackNode;

stackNode* top;

// 스택이 공백인지 확인하는 연산
int isEmpty()
{
    if (top == NULL)
        return 1;
    else
        return 0;
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
void push(int item)
{
    stackNode* temp = (stackNode *)malloc(sizeof(stackNode));
    temp->data = item;
    temp->link = top;
    top = temp;
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
int pop()
{
    int item;
    stackNode* temp = top;

    if (isEmpty()) {
        printf("Stack is empty !\n");
        return 0;
    }
    else {
        item = temp->data;
        top = temp->link;
        free(temp);
        return item;
    }
}
```



## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 깊이 우선 탐색을 위해 초기 공백 그래프를 생성하는 연산
void createGraph(graphType* g)
{
    int v;
    g->n = 0; // 그래프의 정점 개수를 0으로 초기화
    for (v = 0; v < MAX_VERTEX; v++) {
        // 그래프의 정점에 대한 배열 visited를 FALSE로 초기화
        g->visited[v] = FALSE;
        // 인접 리스트에 대한 헤드 노드 배열을 NULL로 초기화
        g->adjList_H[v] = NULL;
    }
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 그래프 g에 정점 v를 삽입하는 연산
void insertVertex(graphType* g, int v)
{
    if (((g->n) + 1) > MAX_VERTEX) {
        printf("\n\n 그래프 정점의 개수를 초과하였습니다!");
        return;
    }
    g->n++;
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 그래프 g에 간선 (u, v)를 삽입하는 연산
void insertEdge(graphType* g, int u, int v)
{
    graphNode* node;
    if (u >= g->n || v >= g->n) {
        printf("₩n 그래프에 없는 정점입니다!");
        return;
    }
    node = (graphNode *)malloc(sizeof(graphNode));
    node->vertex = v;
    node->link = g->adjList_H[u];
    g->adjList_H[u] = node;
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 그래프 g의 각 정점에 대한 인접 리스트를 출력하는 연산
void print_adjList(graphType* g)
{
    int i;
    graphNode* p;
    for (i = 0; i < g->n; i++) {
        printf("WnWtWt정점%c의 인접 리스트", i + 65);
        p = g->adjList_H[i];
        while (p) {
            printf(" -> %c", p->vertex + 65);
            p = p->link;
        }
    }
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 그래프 g에서 정점 v에 대한 깊이 우선 탐색 연산
void DFS_adjList(graphType* g, int v)
{
    graphNode* w;
    top = NULL;    // 스택의 top 설정
    push(v);       // 깊이 우선 탐색을 시작하는 정점 v를 스택에 push
    // 정점 v를 방문했으므로 해당 배열 값을 TRUE로 설정
    g->visited[v] = TRUE;
    printf(" %c", v + 65);

    // 스택이 공백이 아닌 동안 깊이 우선 탐색 반복
    while (!isEmpty()) {
        w = g->adjList_H[v];
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
// 인접 정점이 있는 동안 수행
while (w) {
    // 현재 정점 w를 방문하지 않은 경우
    if (!g->visited[w->vertex]) {
        push(w->vertex);      // 현재 정점 W를 스택에 push
        // 정점 w에 대한 배열 값을 TRUE로 설정
        g->visited[w->vertex] = TRUE;
        // 정점 0~6을 A~G로 바꾸어서 출력
        printf(" %c", w->vertex + 65);
        v = w->vertex;
        w = g->adjList_H[v];
    }
    // 현재 정점 w가 이미 방문된 경우
    else w = w->link;
}
// 현재 정점에서 순회를 진행할 인접 정점이 더 없는 경우에 스택 pop!
v = pop();
} // 스택이 공백이면 깊이 우선 탐색 종료
}
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
void main()
{
    int i;
    graphType *G9;
    G9 = (graphType *)malloc(sizeof(graphType));
    createGraph(G9);

    // 그래프 G9 구성
    for (i = 0; i < 7; i++)
        insertVertex(G9, i);
    insertEdge(G9, 0, 2);
    insertEdge(G9, 0, 1);
    insertEdge(G9, 1, 4);
    insertEdge(G9, 1, 3);
    insertEdge(G9, 1, 0);
    insertEdge(G9, 2, 4);
    insertEdge(G9, 2, 0);
    insertEdge(G9, 3, 6);
    insertEdge(G9, 3, 1);
```

## ❖ 그래프 깊이 우선 탐색하기 프로그램

```
insertEdge(G9, 4, 6);
insertEdge(G9, 4, 2);
insertEdge(G9, 4, 1);
insertEdge(G9, 5, 6);
insertEdge(G9, 6, 5);
insertEdge(G9, 6, 4);
insertEdge(G9, 6, 3);
printf("\n G9의 인접 리스트 ");
print_adjList(G9);

printf("\n\n////////////////\n\n깊이 우선 탐색 >> ");
DFS_adjList(G9, 0);    // 0번 정점인 정점 A에서 깊이 우선 탐색 시작

getchar();
}
```



## ❖ 그래프 깊이 우선 탐색하기 프로그램 : 교재 444p

### ■ 실행 결과

```
C:\WINDOWS\system32\cmd.exe

G9의 인접 리스트
정점A의이전점리스트 -> B -> C
정점B의이전점리스트 -> A -> D -> E
정점C의이전점리스트 -> A -> E
정점D의이전점리스트 -> B -> G
정점E의이전점리스트 -> B -> C -> G
정점F의이전점리스트 -> G
정점G의이전점리스트 -> D -> E -> F

//////////

깊이 우선 탐색 >> A B D G E C F
```